



TRAILHEAD
TECHNOLOGY PARTNERS

Oops, I Leaked It Again

API Security Mistakes Fixed



Jonathan "J." Tower

{
"celebrities":
 "Tom Hanks",
 "Beyoncé",
 "Leonardo DiCaprio",
 "Taylor Swift",
 "Dwayne Johnson",
 "Scarlett Johansson"
}



Most common API
security mistakes,
and **practical fixes** to keep
your REST APIs safe



TRAILHEAD
TECHNOLOGY PARTNERS

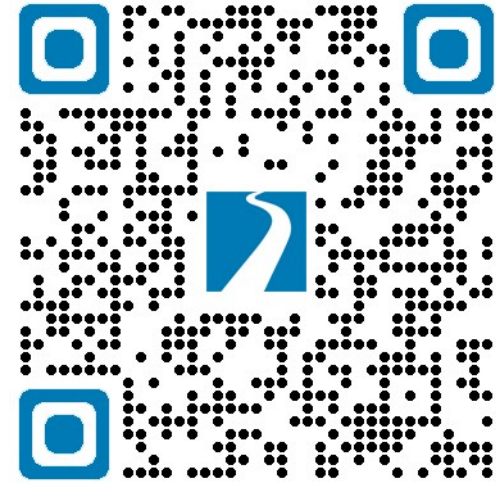
Jonathan "J." Tower

Founding Partner



- 12x Microsoft MVP in .NET
- .NET Foundation Board of Directors
- jtower@trailheadtechnology.com
- trailheadtechnology.com/blog
- [jtowermi](#)
- Jonathan "J." Tower

LET'S TALK

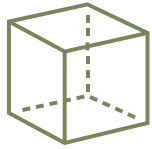


bit.ly/th-offer

github.com/trailheadtechnology/api-security

8 Most Common API Security Mistakes

8 Common Issues



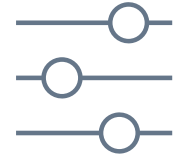
Broken Object
Level Authz



Broken Object
Prop Level Authz



Broken
Authentication



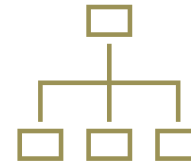
Misconfig &
Insecure Defaults



Unrestricted Resource
Consumption



Verbose Errors or
Logging Leaks



Unrestricted Resource
Consumption



Transport Layer &
Secrets Mgmt

OWASP Top Ten

- Main
- Translation Efforts
- Sponsors
- Data 2025

Important note:

OWASP Top Ten 2025

Current project status as of Sept 2025:

- We are on track to announce the release of the **OWASP Top 10:2025** at the OWASP Global AppSec Conf in DC the first week of Nov 2025.

[Stay Tuned!](#)

The OWASP Top Ten Community Survey is open, please contribute your insights! [Community Survey](#)

The OWASP Top 10 is a standard awareness document for developers and web application security. It represents a broad consensus about the most critical security risks to web applications.

Globally recognized by developers as the first step towards more secure coding.



The OWASP® Foundation works to improve the security of software through its community-led open source software projects, hundreds of chapters worldwide, tens of thousands of members, and by hosting local and global conferences.

Project Information

- [OWASP Top 10:2021](#)
- [Making of OWASP Top 10](#)
- [OWASP Top 10:2021 - 20th Anniversary Presentation \(PPTX\)](#)
- 🚩 [Flagship Project](#)
- 📖 [Documentation](#)
- 🔧 [Builder](#)
- 🛡 [Defender](#)
- [Previous Version \(2017\)](#)

Downloads or Social Links

- [OWASP Top 10 2017](#)
- [Other languages](#) → tab 'Translation Efforts'

Examples



.NET

Examples



.NET



GO

JS

Broken Object Level Authorization

Common Mistake #1

Broken Object Level Authorization

```
//[Authorize]  
[HttpGet("{id:guid}")]  
public Task<User?> GetById(Guid id) => _db.Users.FindAsync(id).AsTask();
```

Broken Object Level Authorization



```
[ApiController]  
[Authorize]  
[Route("api/[controller]")]  
public abstract class BaseApiController : ControllerBase { ... }
```

Broken Object Level Authorization



```
public sealed class OnlyBaseApiControllerFeatureProvider
    : IApplicationFeatureProvider<ControllerFeature>
{
    public void PopulateFeature(IEnumerable<ApplicationPart> parts, ControllerFeature feature)
    {
        var allowed = feature.Controllers
            .Where(t => typeof(BaseApiController).IsAssignableFrom(t.AsType()))
            .ToList();

        var rejected = feature.Controllers.Except(allowed).ToList();
        feature.Controllers.Clear();
        foreach (var c in allowed) feature.Controllers.Add(c);

        if (rejected.Count > 0)
            throw new InvalidOperationException(
                $"Controllers must inherit {nameof(BaseApiController)}. Offenders: " +
                string.Join(", ", rejected.Select(r => r.FullName)));
    }
}
```

Broken Object Level Authorization



```
builder.Services.AddControllers()  
    .ConfigureApplicationPartManager(pm =>  
        pm.FeatureProviders.Add(new OnlyBaseApiControllersFeatureProvider()));
```


Broken Object Level Authorization

```
// Client posts userId/tenantId; server uses it directly
public async Task<IResult> Update(Guid id, UpdateOrderDto dto)
{
    var order = await db.Orders.SingleOrDefaultAsync(o => o.Id == id
        && o.OwnerUserId == dto.UserId); //  client reports user
    // ...
}
```

Broken Object Level Authorization



```
public async Task<IResult> Update(Guid id, UpdateOrderDto dto,
    ClaimsPrincipal user)
{
    var userId = user.FindFirst("sub")?.Value!;

    var order = await db.Orders.SingleOrDefaultAsync(o => o.Id == id
        && o.OwnerUserId == userId);
    // ...
}
```

Broken Object Property Level Authorization

Common Mistake #2

Broken Object Property Level Authorization

Return Value



```
{  
  "id": 123,  
  "username": "jdoe",  
  "email": "jdoe@example.com",  
  "passwordHash": "5f4dcc3b5aa765d61d8327deb882cf99",  
  "ssn": "123-45-6789",  
  "phoneNumber": "2125551212",  
  "isAdmin": false,  
  "lastLogin": "2025-09-25T13:45:00Z",  
  "profilePicture": "iVBORw0KGgoUhEUgAADhCAYAAAAA..."  
}
```

Broken Object Property Level Authorization

Secrets
and
Credentials

```
{  
  "id": 123,  
  "username": "jdoe",  
  "email": "jdoe@example.com",  
  "passwordHash": "5f4dcc3b5aa765d61d8327deb882cf99",  
  "ssn": "123-45-6789",  
  "phoneNumber": "2125551212",  
  "isAdmin": false,  
  "lastLogin": "2025-09-25T13:45:00Z",  
  "profilePicture": "iVBORw0KGgoUhEUgAADhCAYAAAAA..."  
}
```

Broken Object Property Level Authorization

Privilege Hints & Implementation Details

```
{  
  "id": 123,  
  "username": "jdoe",  
  "email": "jdoe@example.com",  
  "passwordHash": "5f4dcc3b5aa765d61d8327deb882cf99",  
  "ssn": "123-45-6789",  
  "phoneNumber": "2125551212",  
  "isAdmin": false,  
  "lastLogin": "2025-09-25T13:45:00Z",  
  "profilePicture": "iVBORw0KGgoUhEUgAADhCAYAAAA..."  
}
```

Broken Object Property Level Authorization

Heavy or
Unused Fields

```
{  
  "id": 123,  
  "username": "jdoe",  
  "email": "jdoe@example.com",  
  "passwordHash": "5f4dcc3b5aa765d61d8327deb882cf99",  
  "ssn": "123-45-6789",  
  "phoneNumber": "2125551212",  
  "isAdmin": false,  
  "lastLogin": "2025-09-25T13:45:00Z",  
  "profilePicture": "iVBORw0KGgoUhEUgAADhCAYAAAAA..."  
}
```

Broken Object Property Level Authorization

Expose These
Thoughtfully

```
{  
  "id": 123,  
  "username": "jdoe",  
  "email": "jdoe@example.com",  
  "passwordHash": "5f4dcc3b5aa765d61d8327deb882cf99",  
  "ssn": "123-45-6789",  
  "phoneNumber": "2125551212",  
  "isAdmin": false,  
  "lastLogin": "2025-09-25T13:45:00Z",  
  "profilePicture": "iVBORw0KGgoUhEUgAADhCAYAAAAA..."  
}
```


Mass Assignment

Mass Assignment

```
public class UpdateUserRequestDto
{
    public Guid Id { get; set; }
    public string Email { get; set; } = "";
    public string DisplayName { get; set; } = "";

    public bool IsAdmin { get; set; }

    public Guid OwnerId { get; set; }
}
```

Mass Assignment



Mass Assignment

```
public class UpdateUserRequestDto
{
    public Guid Id { get; set; }
    public string Email { get; set; } = "";
    public string DisplayName { get; set; } = "";

    [System.Text.Json.Serialization.JsonIgnore]
    public bool IsAdmin { get; set; }

    [Microsoft.AspNetCore.Mvc.ModelBinding.BindNever]
    public Guid OwnerId { get; set; }
}
```

Mass Assignment



Mass Assignment

```
public class UpdateUserRequestDto
{
    public Guid Id { get; set; }
    public string Email { get; set; } = "";
    public string DisplayName { get; set; } = "";

    public bool IsAdmin { get; set; }

    public Guid OwnerId { get; set; }
}
```

Treat APIs Like UIs

```
{  
  "id": 123,  
  "Name": "Person Name"  
  "isVerified": true  
}
```

ID:

Name:

Verified:



Treat APIs Like UIs

```
{  
  "id": 123,  
  "Name": "Person Name"  
  "isVerified": true  
}
```

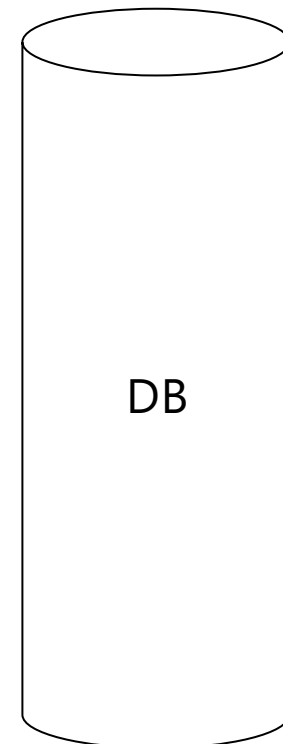
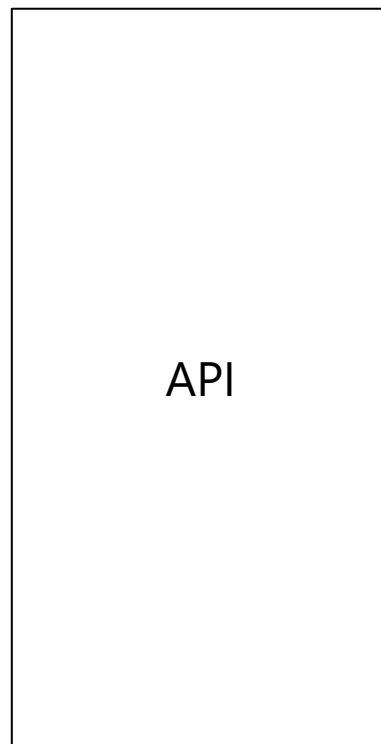
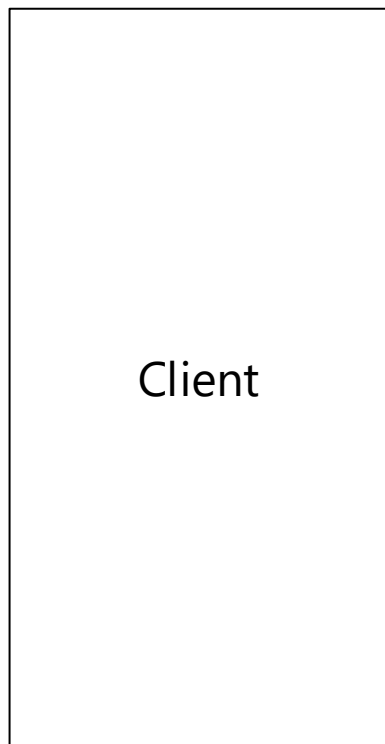
ID:

Name:

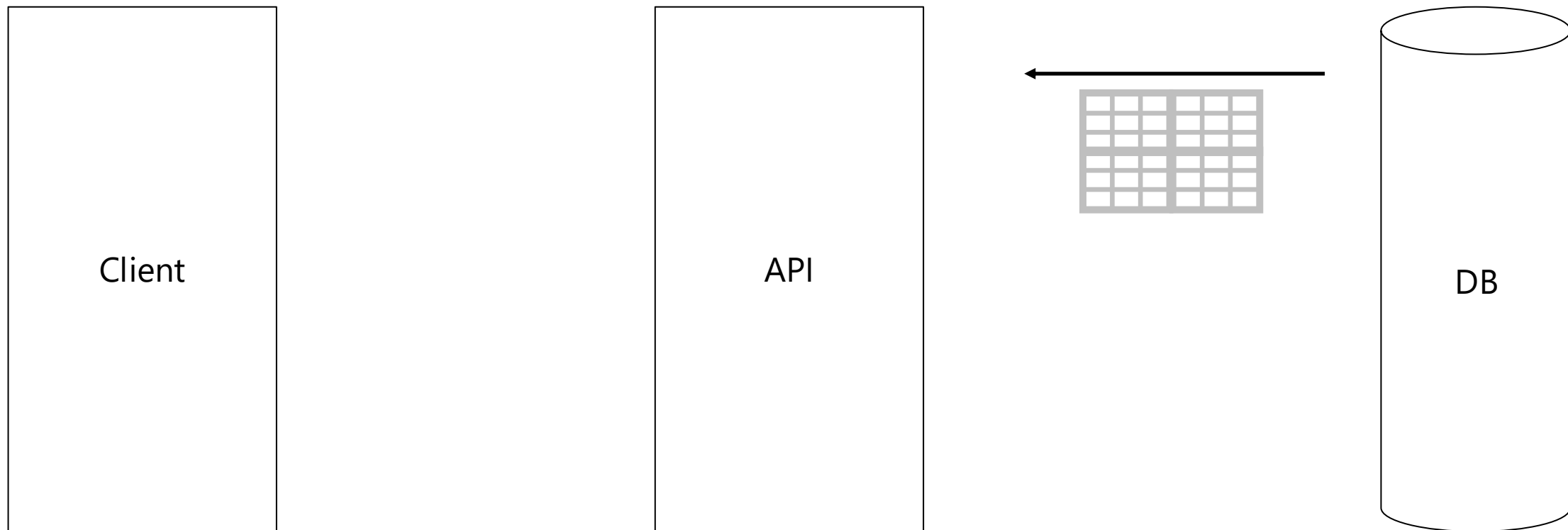
Verified:



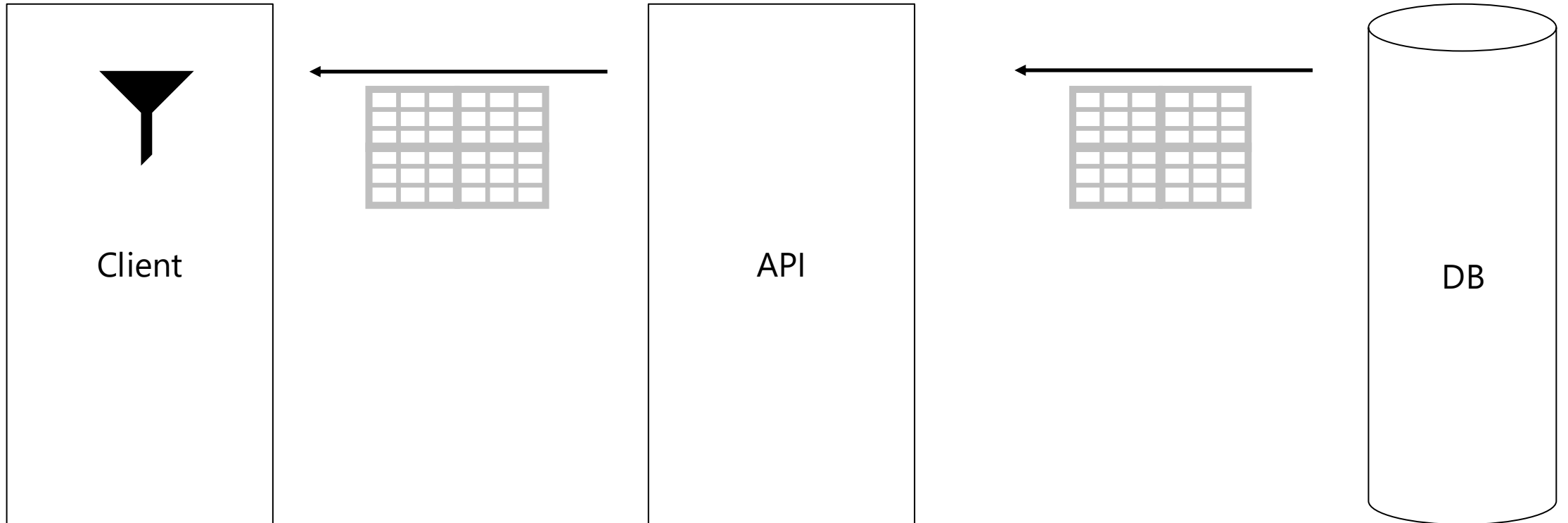
Use DTOs / ViewModels



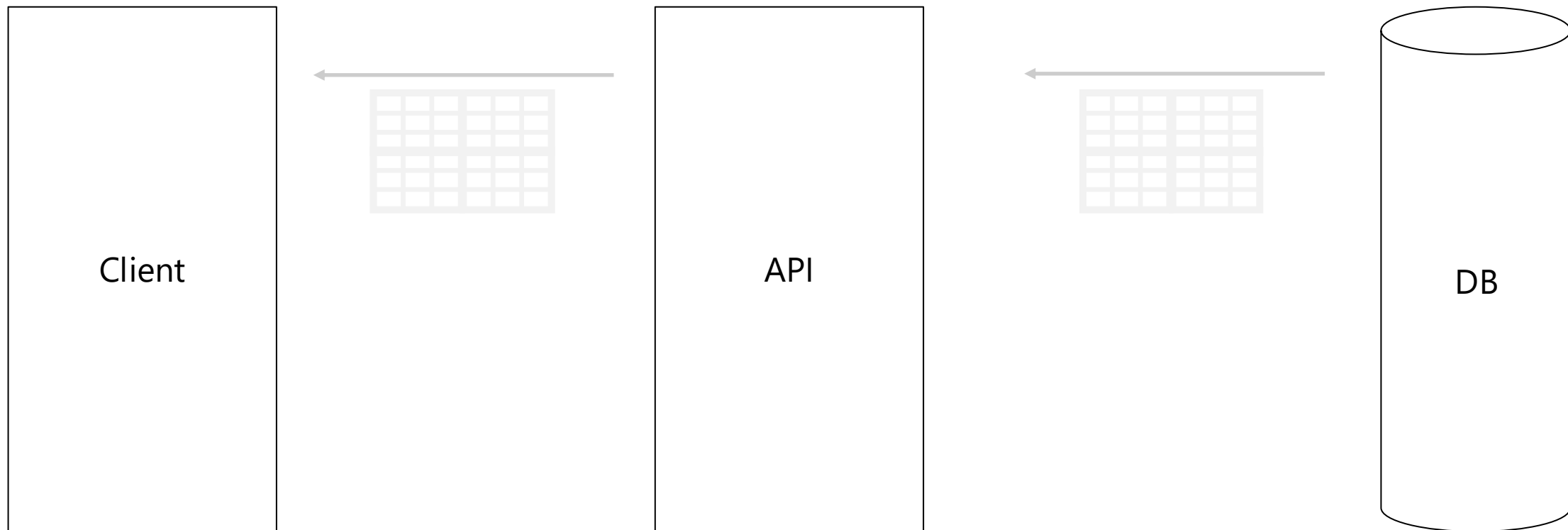
Use DTOs / ViewModels



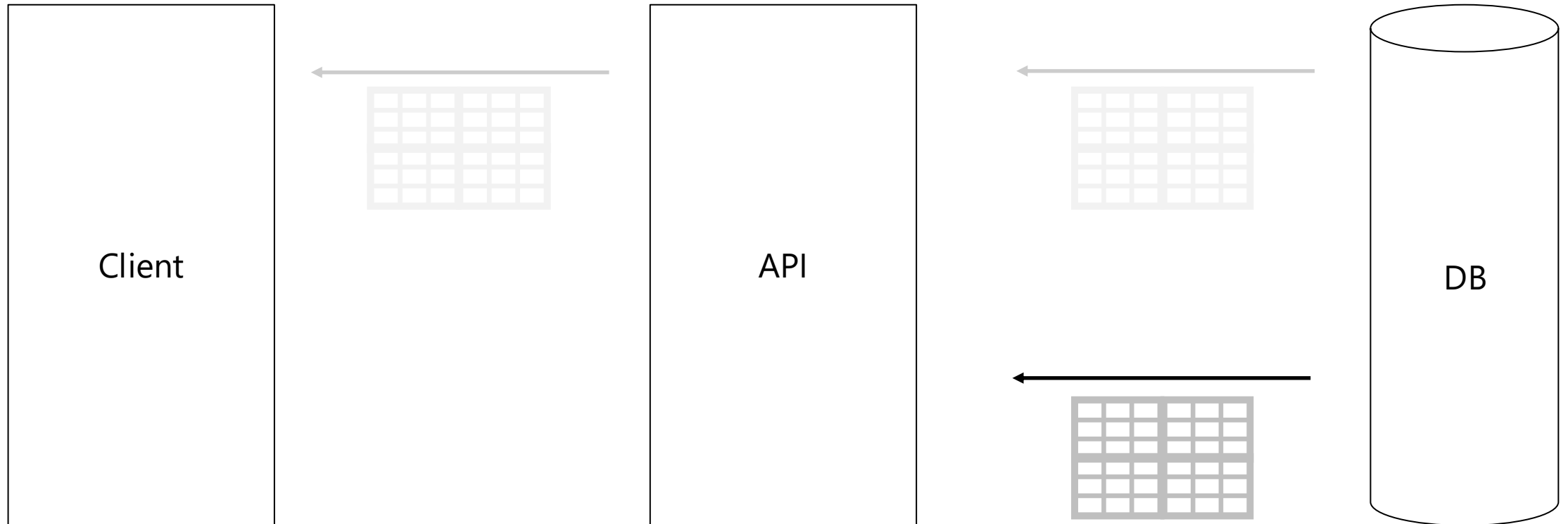
Use DTOs / ViewModels



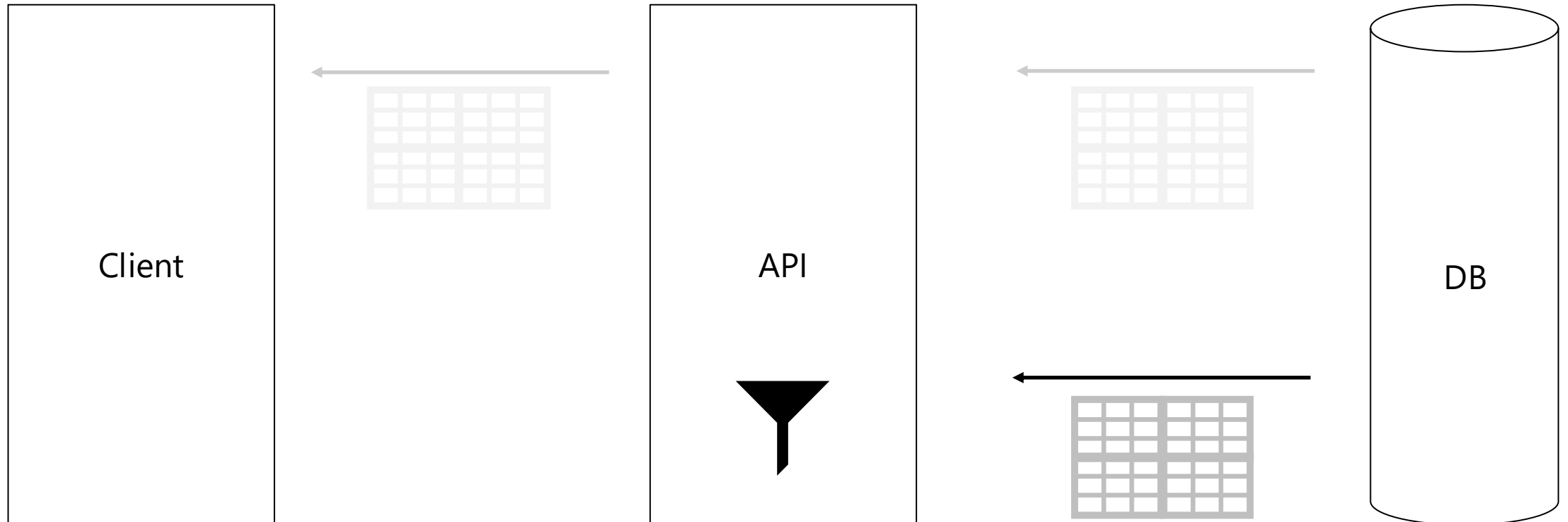
Use DTOs / ViewModels



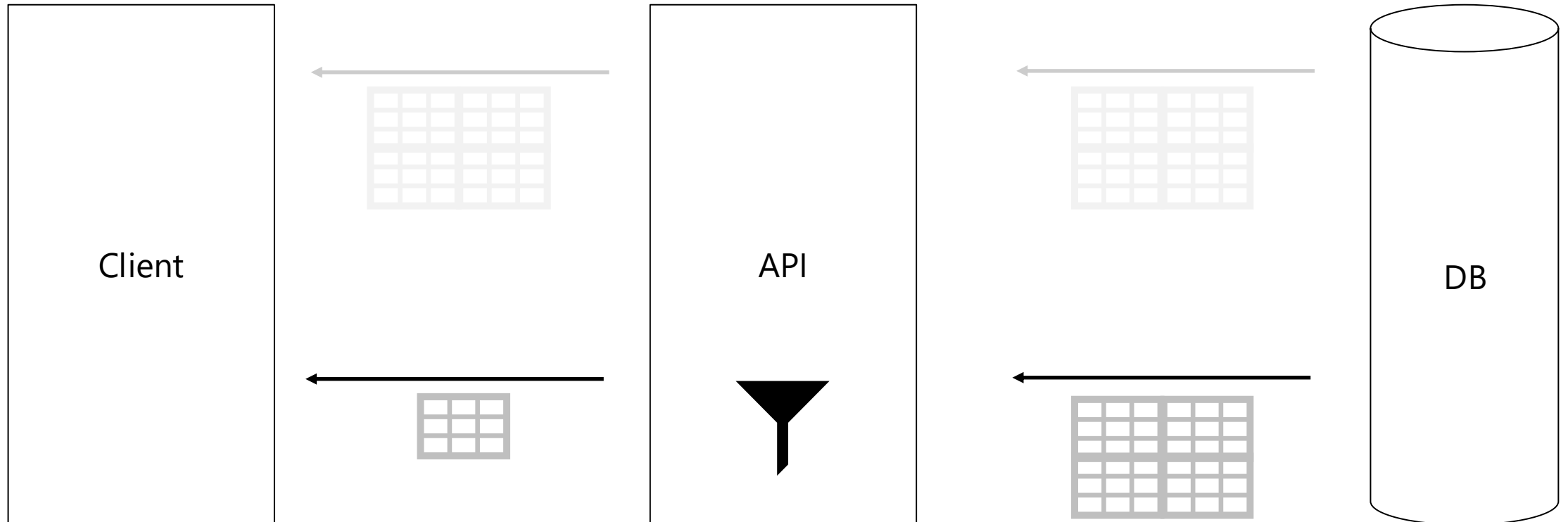
Use DTOs / ViewModels



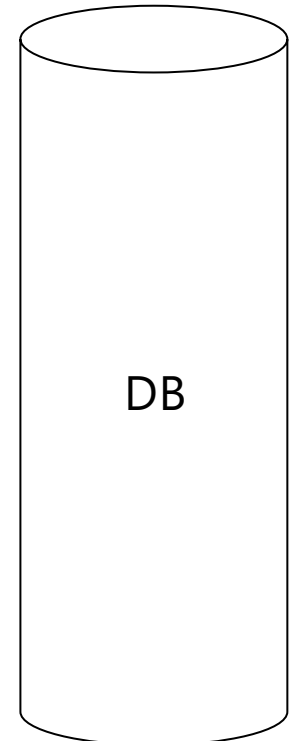
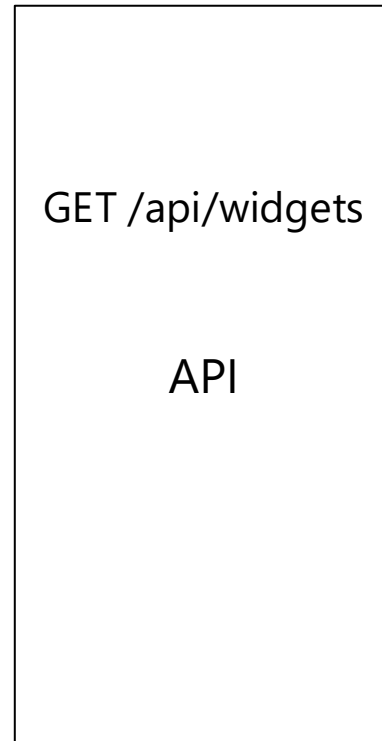
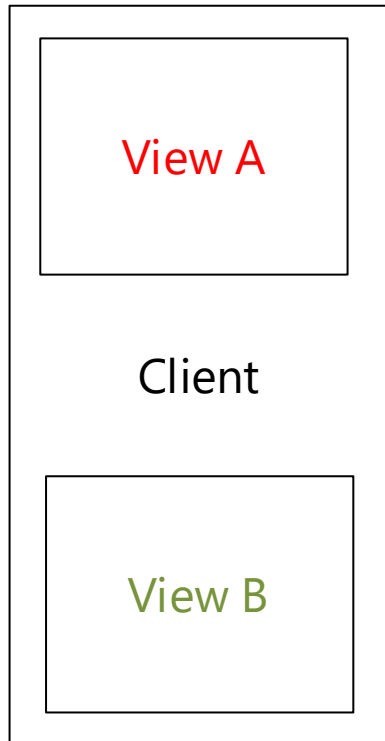
Use DTOs / ViewModels



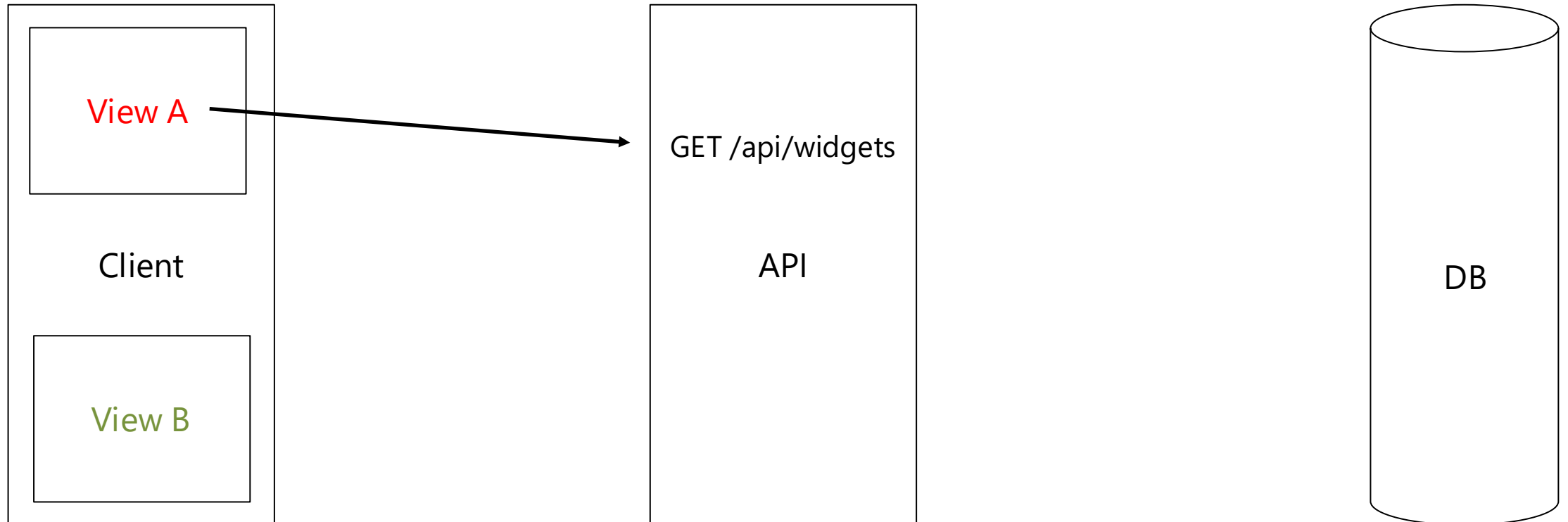
Use DTOs / ViewModels



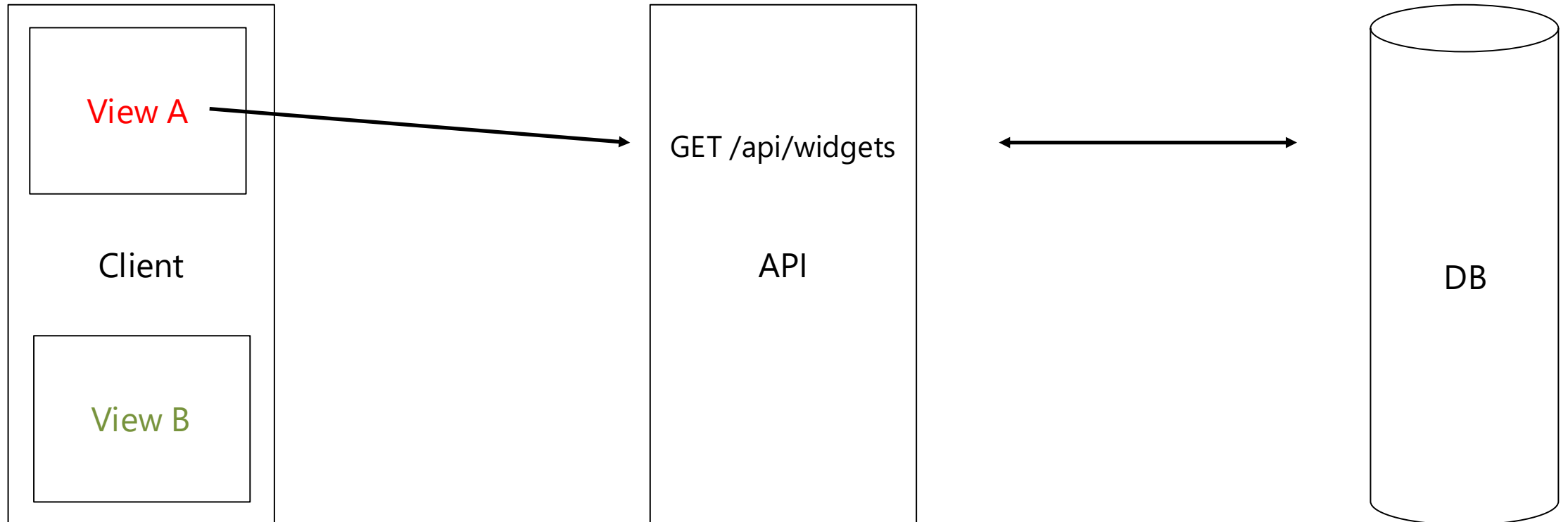
Avoid General APIs



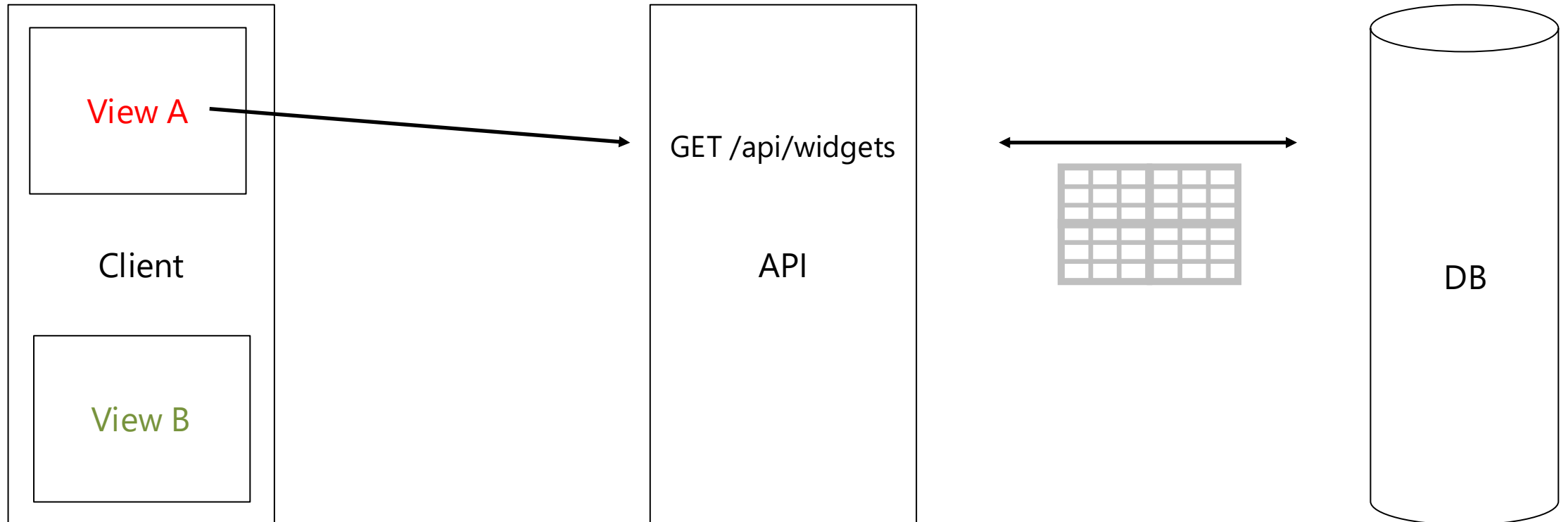
Avoid General APIs



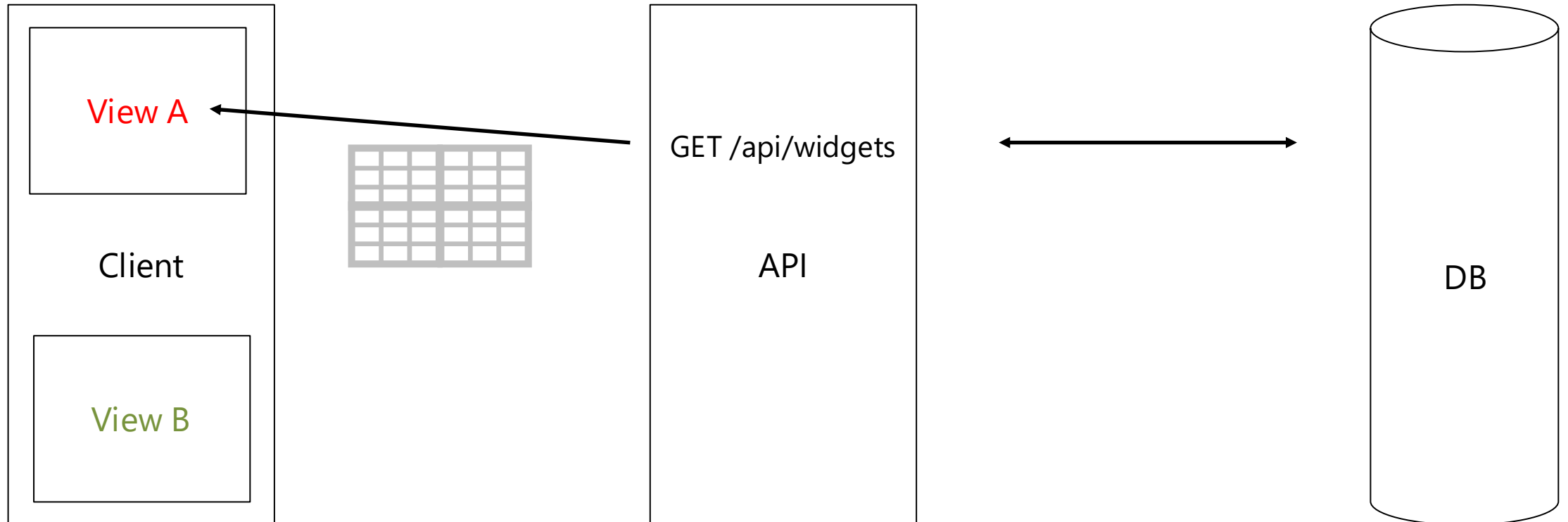
Avoid General APIs



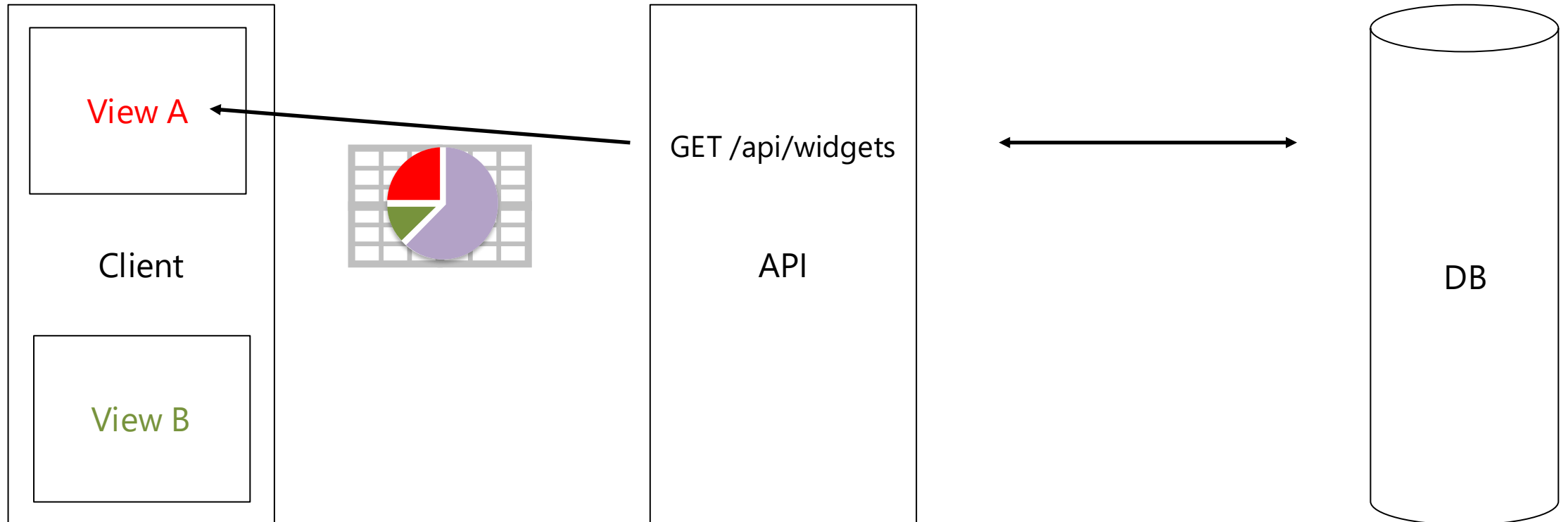
Avoid General APIs



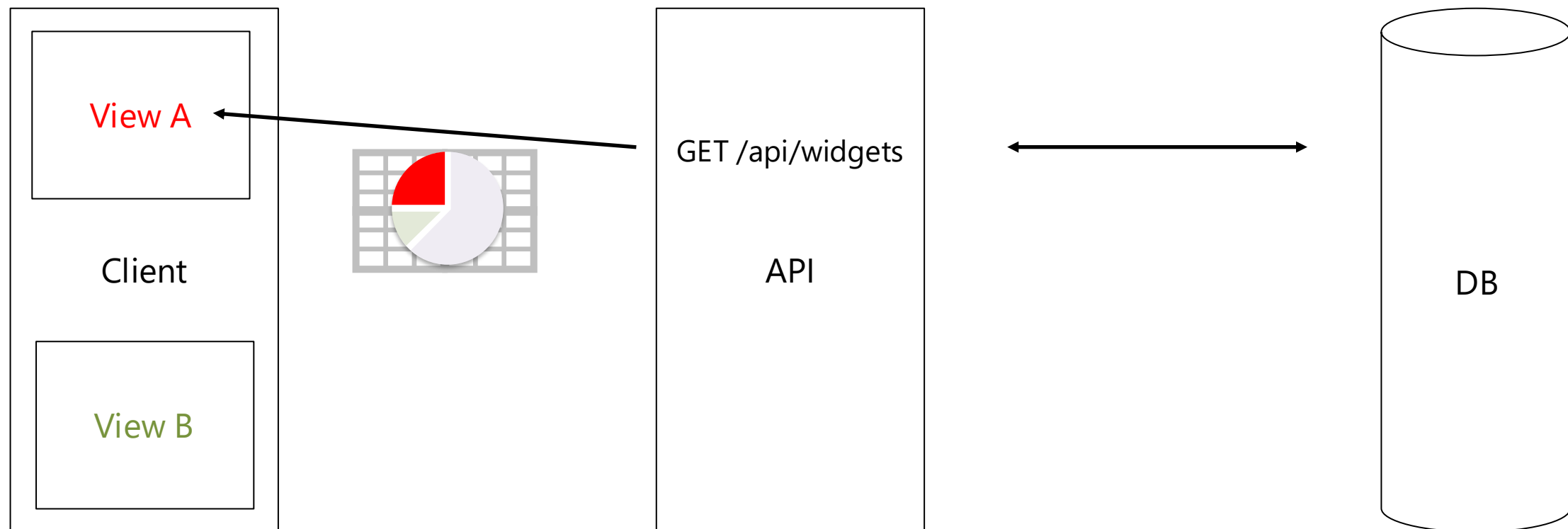
Avoid General APIs



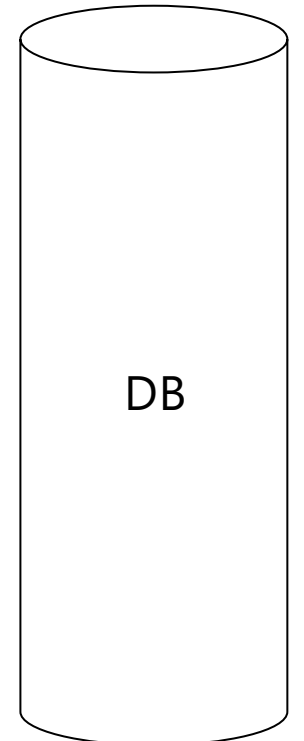
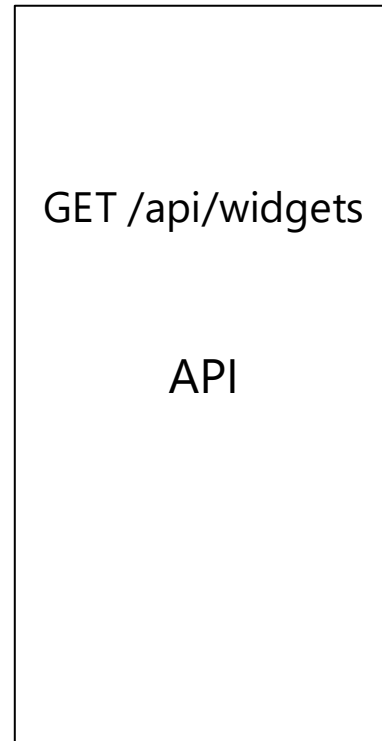
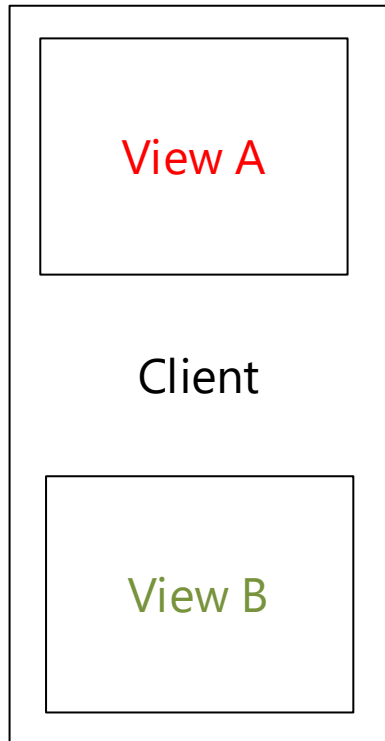
Avoid General APIs



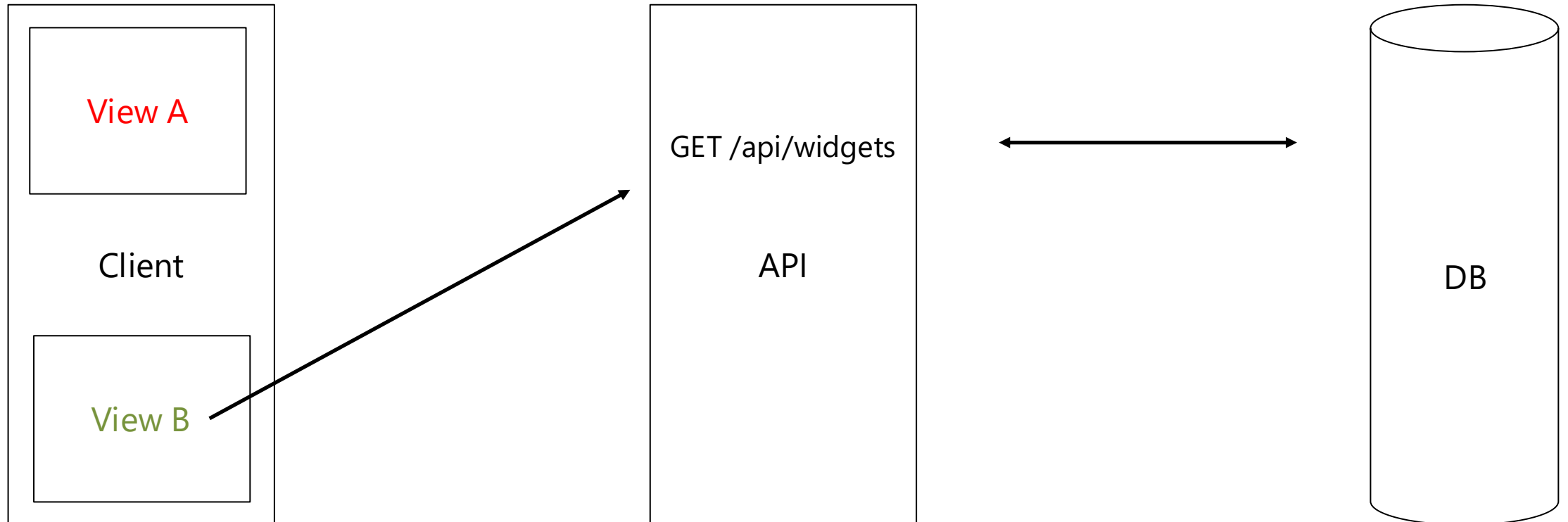
Avoid General APIs



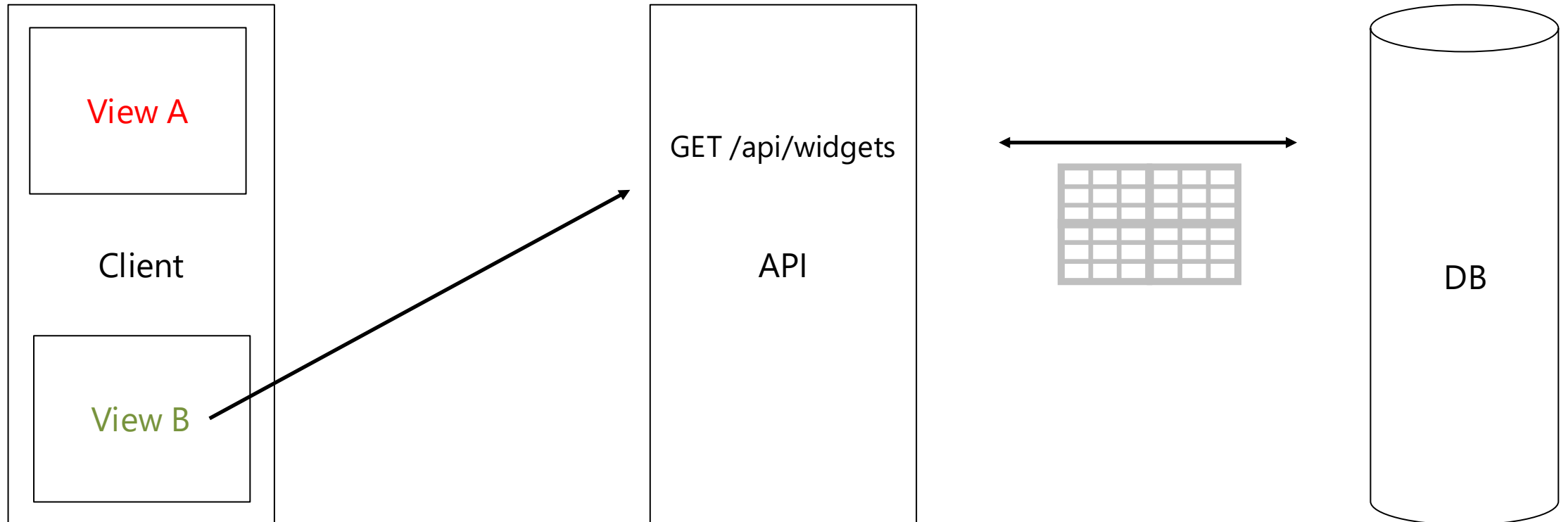
Avoid General APIs



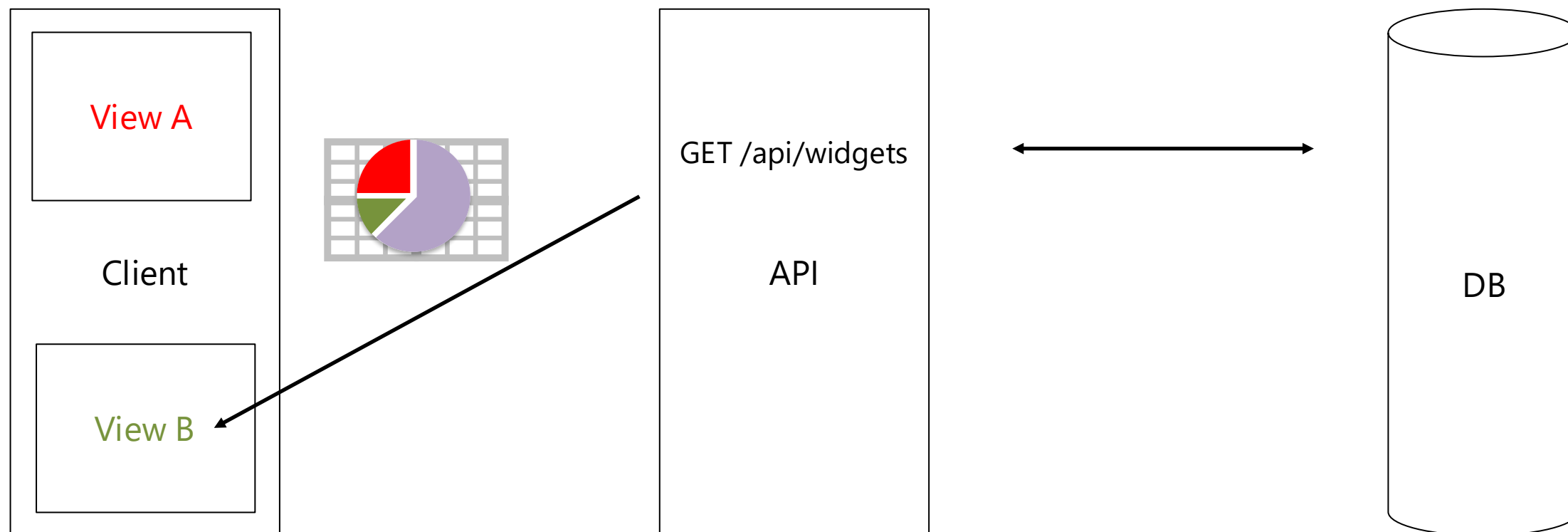
Avoid General APIs



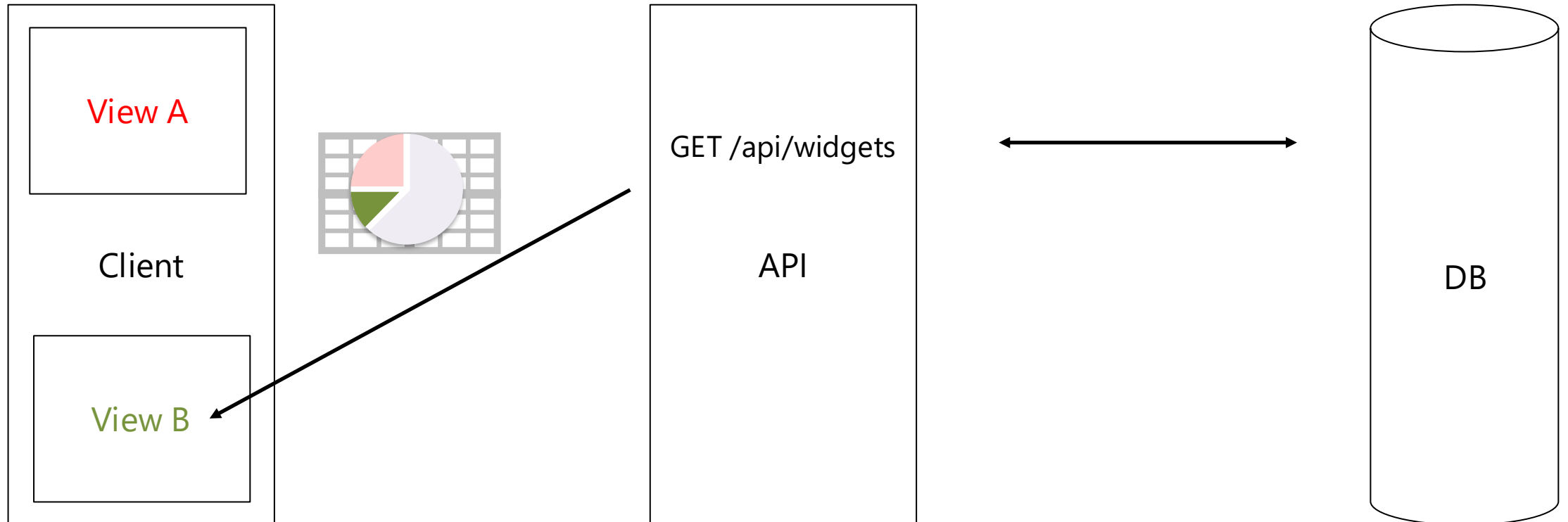
Avoid General APIs



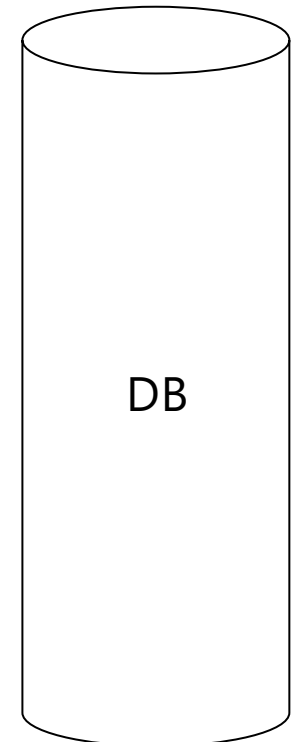
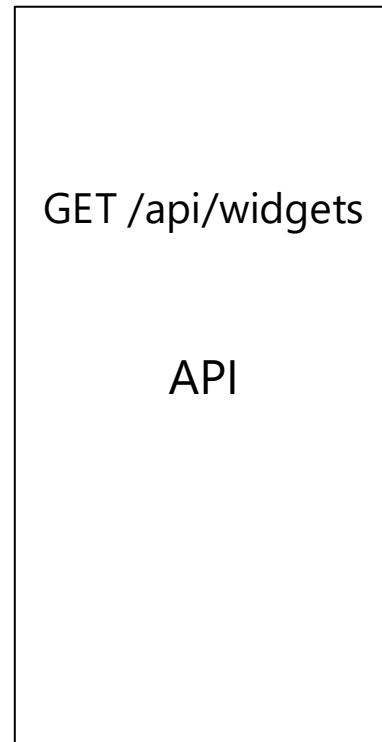
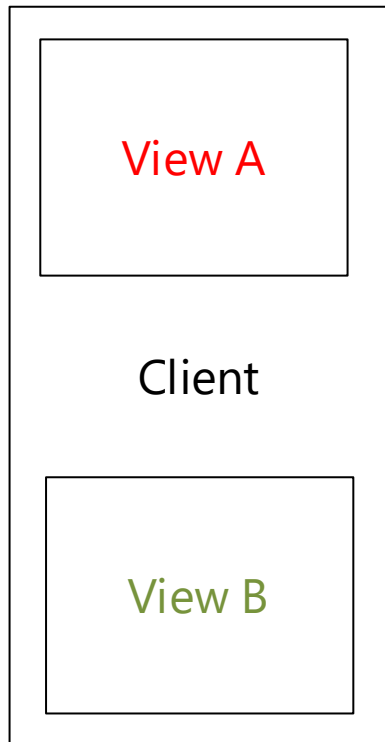
Avoid General APIs



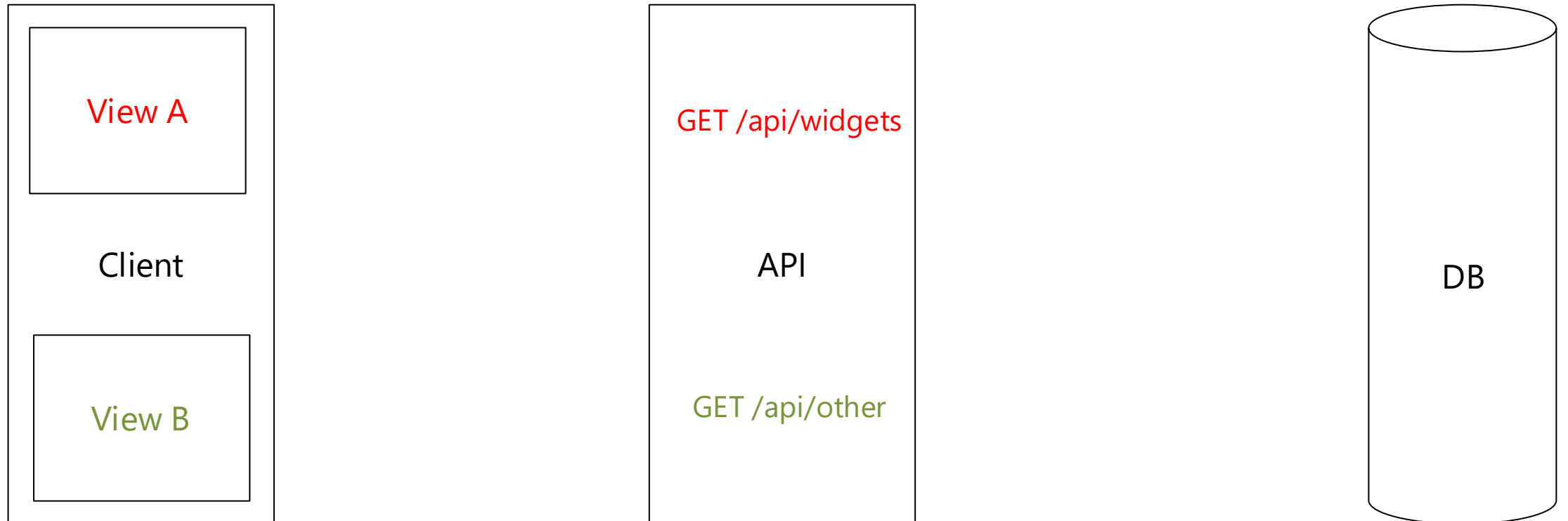
Avoid General APIs



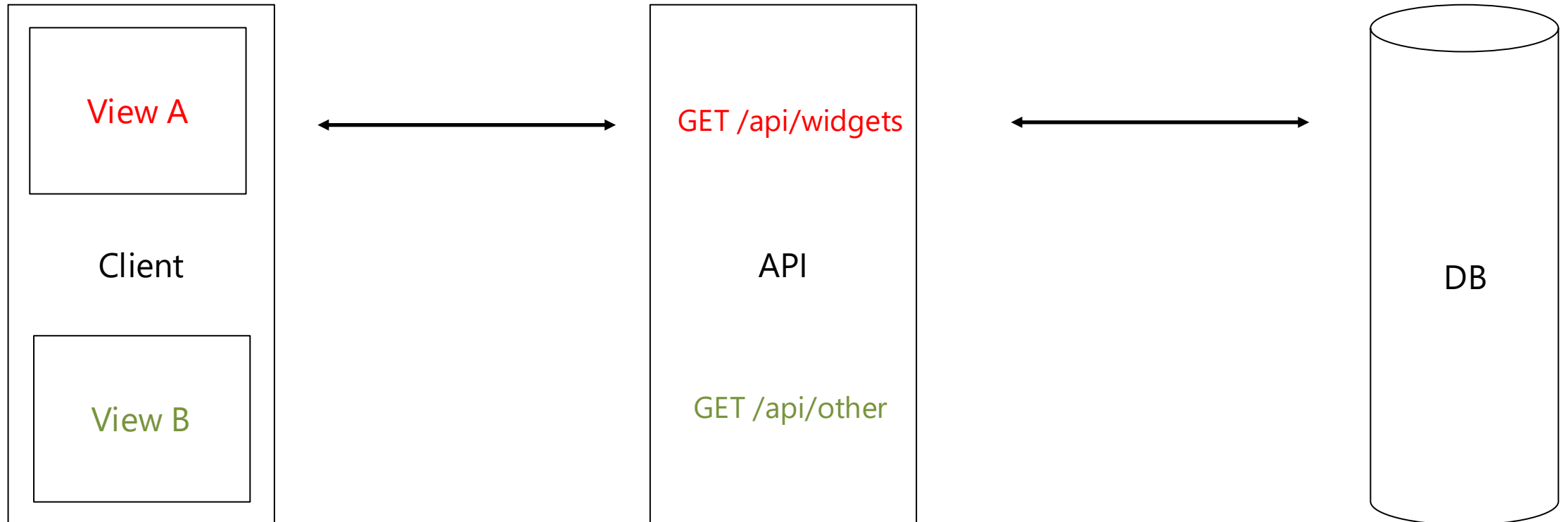
Avoid General APIs



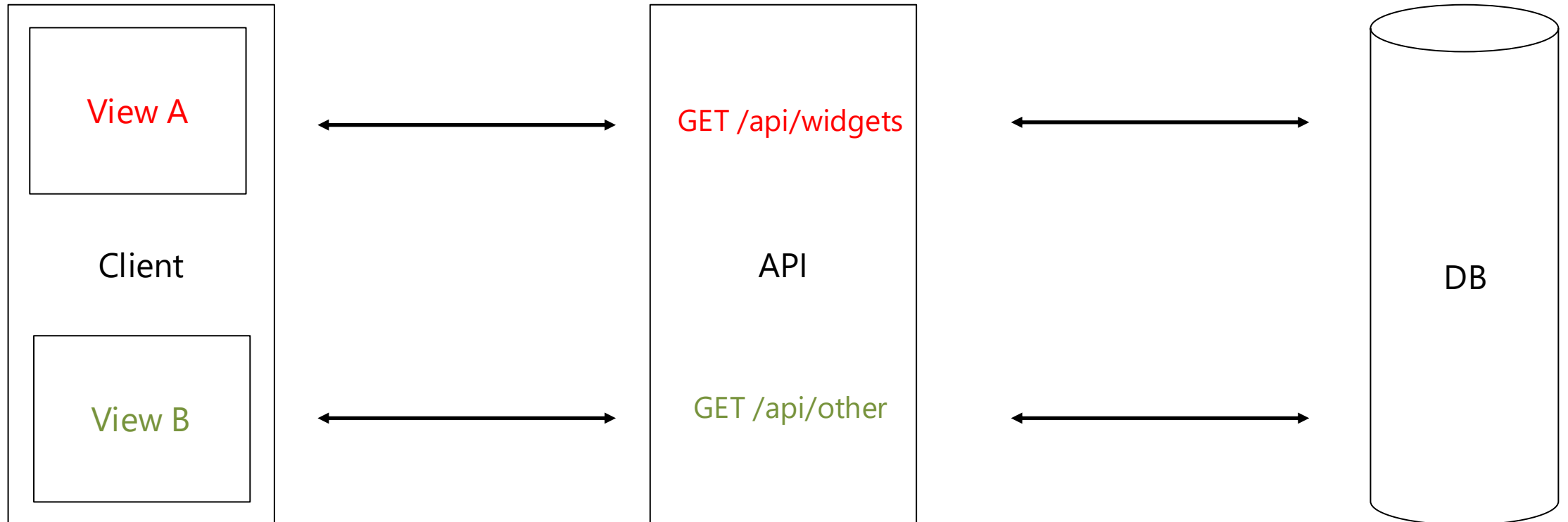
Avoid General APIs



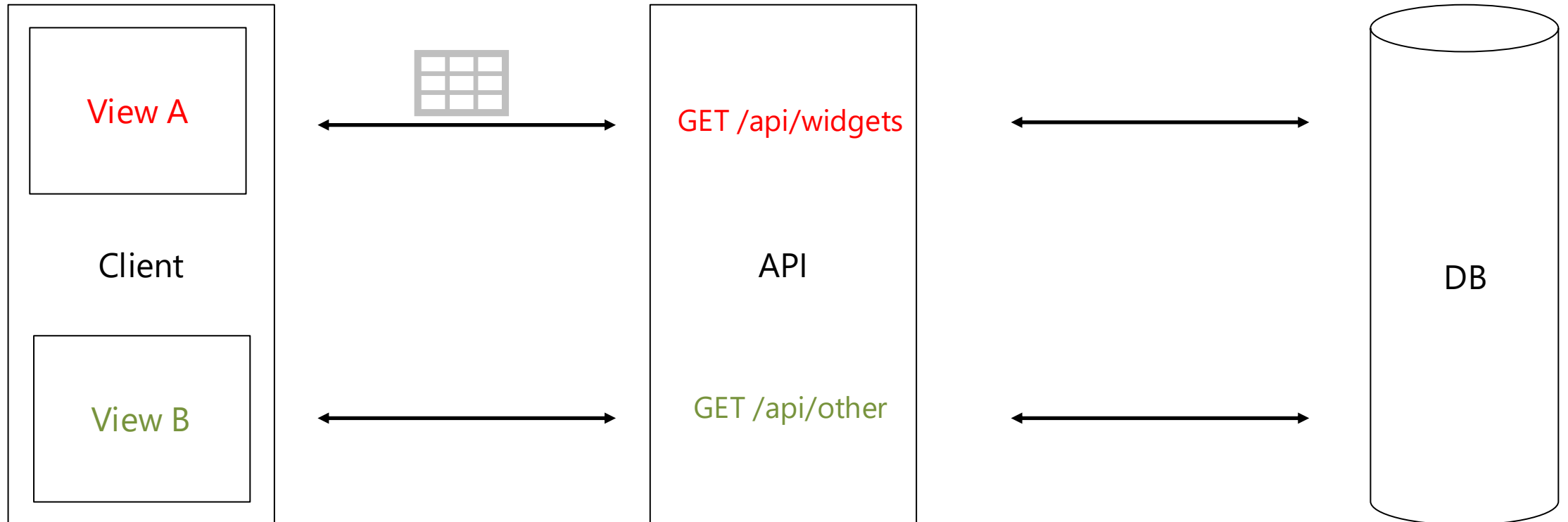
Avoid General APIs



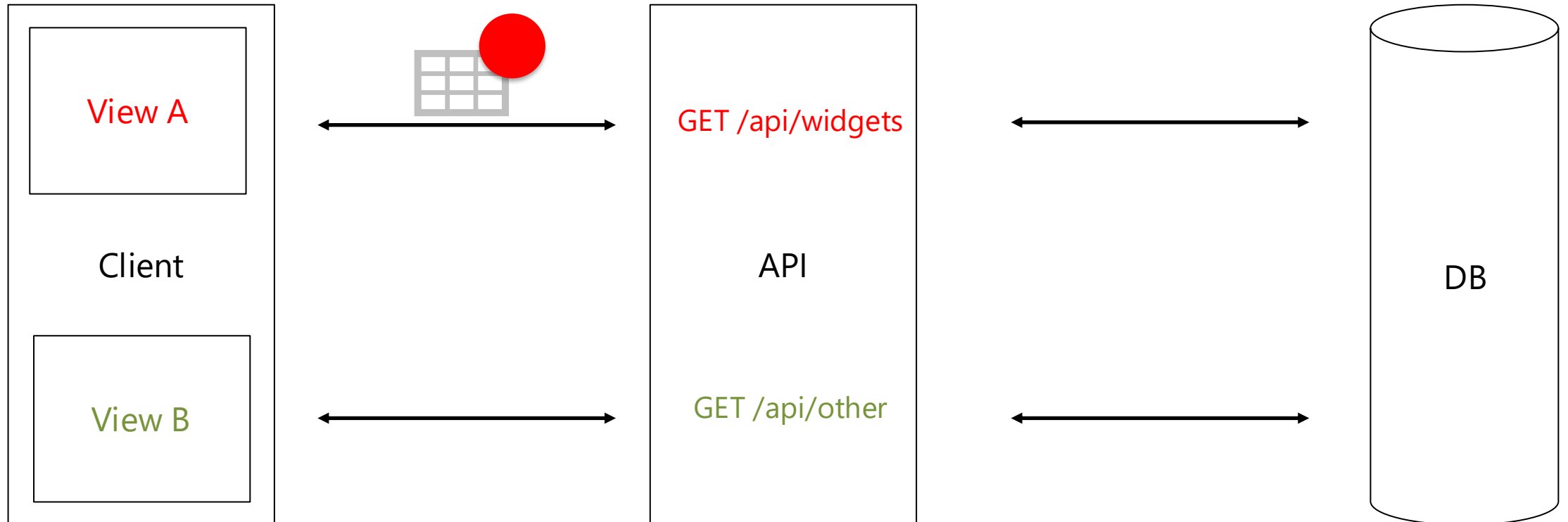
Avoid General APIs



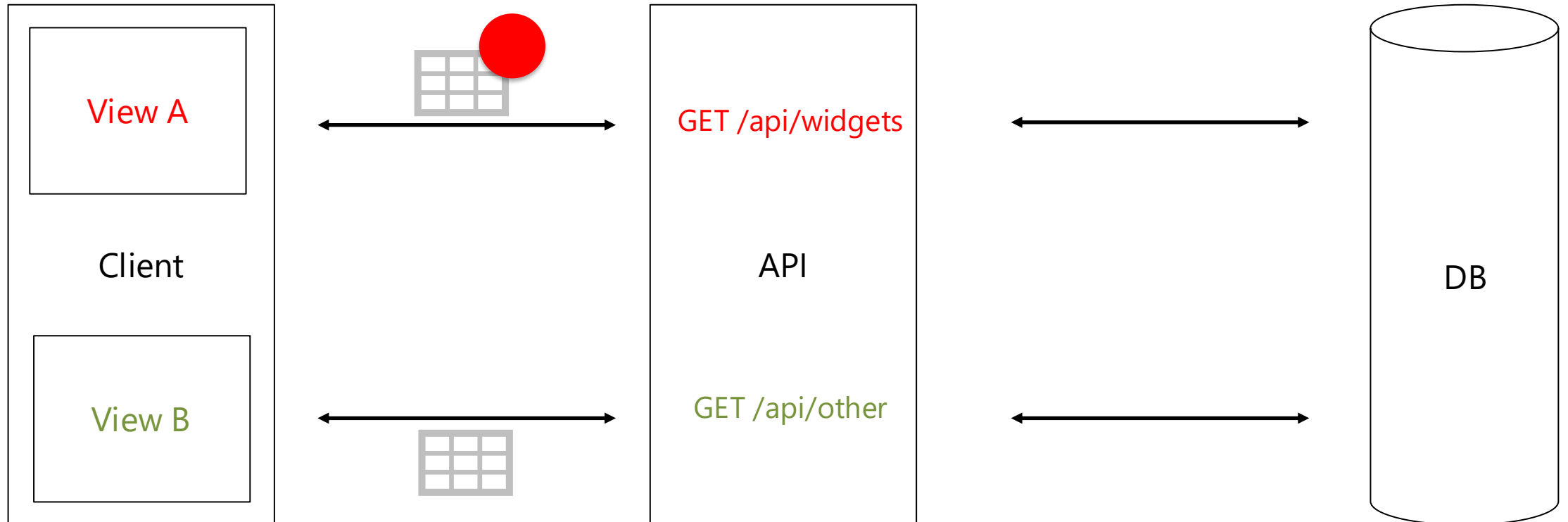
Avoid General APIs



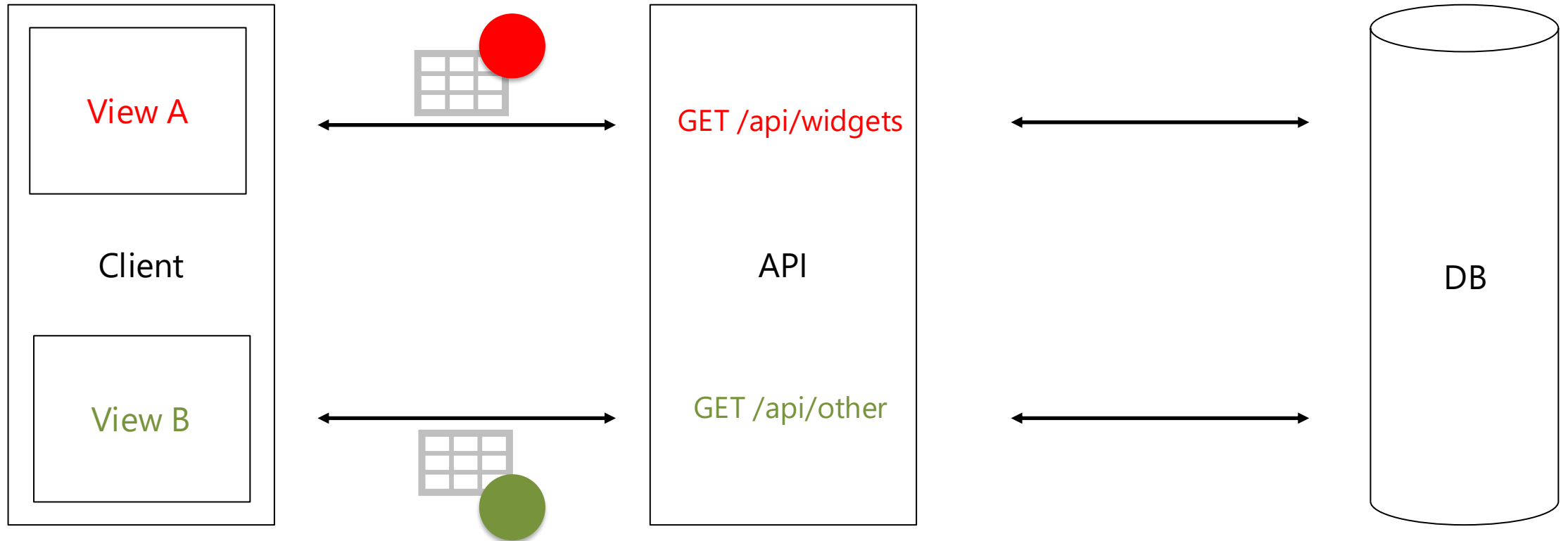
Avoid General APIs



Avoid General APIs



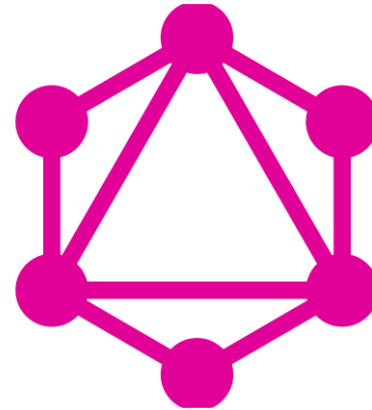
Avoid General APIs



Avoid General APIs ❌



OData



GraphQL

Large Data Handling



```
{  
  ...  
  "profilePicture": "iVBORw0KGgoUgAADhCAYAAAAA..."  
  ...  
}
```



```
{  
  ...  
  "profilePictureUri": "https://img.domain.com/4AF..."  
  ...  
}
```

Large Data Handling



```
{  
  ...  
  "profilePicture": "iVBORw0KGgoUgAADhCAYAAAAA..."  
  ...  
}
```



```
{  
  ...  
  "profilePictureUri": "https://img.domain.com/4F0..."  
  ...  
}
```

Broken Authentication

Common Mistake #3

Weak Signing Key

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(o =>
    {
        o.TokenValidationParameters = new TokenValidationParameters
        {
            ...
            IssuerSigningKey = "secret123", //  weak
            ...
        };
    });
```

Core Token Checks Disabled



```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(o =>
    {
        o.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = false,           // ✗ off
            ValidateAudience = false,        // ✗ off
            ValidateLifetime = false,         // ✗ off
            ValidateIssuerSigningKey = false // ✗ key not checked
            // (no ValidAlgorithms set)
        };
    });
```

Mixed Environment Configuration



Environment	SigningKey	Audience	Issuer
Development	NmQ5NjI2YWYtMDUyNi00MmL...	AppUser	AppName
Testing	NmQ5NjI2YWYtMDUyNi00MmL...	AppUser	AppName
Staging	NmQ5NjI2YWYtMDUyNi00MmL...	AppUser	AppName
Production	NmQ5NjI2YWYtMDUyNi00MmL...	AppUser	AppName

Mixed Environment Configuration



Environment	SigningKey	Audience	Issuer
Development	NmQ5NjI2YWYtMDUyNi00MmL...	AppUser-dev	https://login-dev.appname.local
Testing	YWViYThiYzltZDI3Ny00MzUzLFk...	AppUser-test	https://login-test.appname.internal
Staging	ZTQyMmM1ZWUtNTNINy00ZjL...	AppUser-staging	https://login-staging.appname.com
Production	OTdiNDdmMjI5NGFINDAxNDMT...	AppUser	https://login.appname.com

Long-lived Access Tokens



- Access tokens last hours or days
- Refresh tokens never rotate

Long-lived Access Tokens



- Short access token TTL (5–15 min)
- Rotate refresh tokens and **revoke the old** on use
- Invalidate all tokens on password reset

Misconfiguration and Insecure Defaults

Common Mistake #4

Security Headers

```
// default kestrel config  
var builder = WebApplication.CreateBuilder(args);
```

Security Headers



```
builder.WebHost.ConfigureKestrel(o => o.AddServerHeader = false);  
app.Use(async (ctx, next) =>  
{  
    ctx.Response.Headers["X-Content-Type-Options"] = "nosniff";  
    ctx.Response.Headers["Referrer-Policy"] = "no-referrer";  
    ctx.Response.Headers["Permissions-Policy"] = "geolocation=()";  
    await next();  
});
```

Proxy and Load Balancer Awareness



```
app.UseForwardedHeaders();
```

Proxy and Load Balancer Awareness



```
app.UseForwardedHeaders(new ForwardedHeadersOptions
{
    ForwardedHeaders = ForwardedHeaders.XForwardedFor |
                      ForwardedHeaders.XForwardedProto |
                      ForwardedHeaders.XForwardedHost,
    KnownProxies = { System.Net.IPAddress.Parse("10.0.0.5") }
});
```

CORS That Is Not Wide Open



```
builder.Services.AddCors(p => p.AddPolicy("open", c => c
    .AllowAnyOrigin()
    .AllowAnyHeader()
    .AllowAnyMethod()
    .AllowCredentials()));
```


CORS That Is Not Wide Open



```
builder.Services.AddCors(p => p.AddPolicy("spa", c => c
    .WithOrigins("https://app.example.com")
    .WithHeaders("Content-Type", "Authorization")
    .WithMethods("GET", "POST", "PUT", "DELETE")
    .AllowCredentials()));
app.UseCors("spa");
```

OpenAPI in Production



```
app.UseSwagger();  
app.UseSwaggerUI();
```

```
app.MapSwagger();
```

```
app.MapHealthChecks("/healthz"); // simple liveness
```

OpenAPI in Production



```
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.MapSwagger().RequireAuthorization("Docs.Read");

app.MapHealthChecks("/healthz"); // simple liveness
```

Cookies



```
.AddCookie(o => {  
    o.Cookie.HttpOnly = false;  
    o.Cookie.SecurePolicy = CookieSecurePolicy.None;  
    o.Cookie.SameSite = SameSiteMode.None;  
});
```

Cookies



```
.AddCookie(o => {  
    o.Cookie.HttpOnly = true;  
    o.Cookie.SecurePolicy = CookieSecurePolicy.Always;  
    o.Cookie.SameSite = SameSiteMode.Lax; // Strict if same-site  
    o.SlidingExpiration = true;  
});
```

JSON Binding and Response Shaping



- Bind entities directly
- Unknown JSON fields ignored

JSON Binding and Response Shaping



```
services.AddControllers().AddJsonOptions(o =>  
    o.JsonSerializerOptions.UnmappedMemberHandling =  
        JsonUnmappedMemberHandling.Disallow);
```

Unrestricted Resource Consumption

Common Mistake #5

Paging and Query Shape



```
// Unbounded read: client controls page size
```

```
[HttpGet] public Task<List<Order>> Get(int page = 1, int pageSize = 5000)  
    => db.Orders.Skip((page-1)*pageSize).Take(pageSize).ToListAsync();
```

Paging and Query Shape



```
const int MaxPageSize = 100; // server-enforced cap
[HttpGet] public async Task<IResult> Get(int page = 1, int pageSize = 50,
CancellationTokens ct = default)
{
    pageSize = Math.Clamp(pageSize, 1, MaxPageSize);
    var items = await db.Orders
        .AsNoTracking()
        .OrderByDescending(o => o.CreatedUtc)
        .Select(o => new OrderDto { Id = o.Id, Total = o.Total })
        .Skip((page-1)*pageSize).Take(pageSize)
        .ToListAsync(ct);
    return Results.Ok(items);
}
```

Rate Limiting



```
// No throttling; login and search endpoints can be hammered  
app.MapPost("/login", LoginHandler);  
app.MapGet("/search", SearchHandler);
```

Rate Limiting



```
builder.Services.AddRateLimiter(o =>
{
    o.RejectionStatusCode = 429;
    o.AddPolicy("tight", ctx => RateLimitPartition.GetTokenBucketLimiter(
        ctx.Connection.RemoteIpAddress?.ToString() ?? "anon",
        _ => new TokenBucketRateLimiterOptions
        {
            TokenLimit = 5,           // max in bucket
            TokensPerPeriod = 5,      // refill each period
            ReplenishmentPeriod = TimeSpan.FromMinutes(1),
            AutoReplenishment = true
        }
    ));
});

...

// Apply the single policy to just the sensitive endpoint(s)
app.UseRateLimiter();
app.MapPost("/login", LoginHandler).RequireRateLimiting("tight");
app.MapGroup("/auth").RequireRateLimiting("tight"); // apply to many at once
```

Request Body Size



Kestrel default is 30 MB

Request Body Size



```
builder.WebHost.ConfigureKestrel(o =>  
    o.Limits.MaxRequestBodySize = 2 * 1024 * 1024); // 2 MB
```

Verbose Errors & Logging Leaks

Common Mistake #6

Error Handling in Production



```
app.UseDeveloperExceptionPage();
```


Error Handling in Production



```
if (app.Environment.IsDevelopment())
{
    app.UseDeveloperExceptionPage();
}
else
{
    app.UseExceptionHandler("/error");

    app.Map("/error", () => Results.Problem(
        statusCode: 500,
        title: "An error occurred")); // keep generic
}
```

ProblemDetails, Not Stack Traces



```
app.MapGet("/boom", () => throw new InvalidOperationException("DB timeout..."));

app.Map("/error", (HttpContext ctx) =>
{
    // Optional: attach a correlation id for support
    var cid = ctx.TraceIdentifier;
    return Results.Problem(
        statusCode: StatusCodes.Status500InternalServerError,
        title: "An error occurred",
        extensions: new Dictionary<string, object?> { ["correlationId"] = cid }
    );
});

app.Run();
```

Redact Secrets In Logs



```
builder.Services.AddHttpLogging(); // ✗ logs lots of headers/body by default
```

```
builder.Logging.AddConsole();
```

```
builder.Logging.SetMinimumLevel(LogLevel.Information);
```

```
optionsBuilder
```

```
    .EnableSensitiveDataLogging() // ✗ logs PII and secrets in SQL params
```

```
    .EnableDetailedErrors()
```

```
    .LogTo(Console.WriteLine, LogLevel.Information); // ✗ broad, unfiltered
```

Redact Secrets In Logs



```
// HTTP logging: safe fields only
b.Services.AddHttpLogging(o =>
    o.LoggingFields = HttpLoggingFields.RequestPath |
                    HttpLoggingFields.RequestMethod |
                    HttpLoggingFields.ResponseStatusCode);

// EF Core: no sensitive data logging; show SQL shape in Dev only
b.Services.AddDbContext<AppDbContext>(opts =>
{
    if (b.Environment.IsDevelopment())
        opts.EnableDetailedErrors()
            .LogTo(Console.WriteLine,
                new[] { DbLoggerCategory.Database.Command.Name },
                LogLevel.Information);
    // Prod: default (no LogTo, no SensitiveDataLogging)
});
```

Redact Secrets In Logs

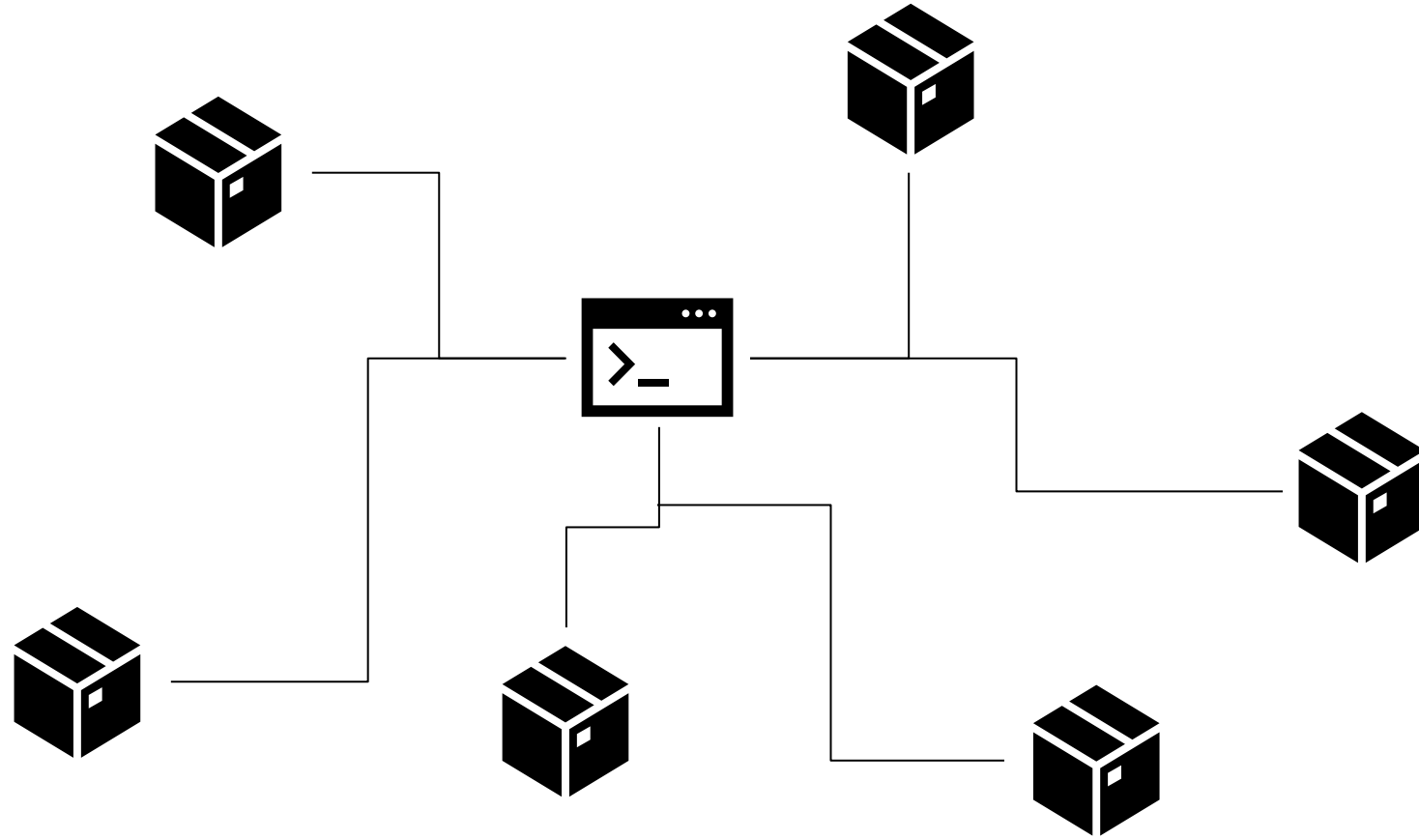


```
// Redact at your logger (example: Serilog)
Log.Logger = new LoggerConfiguration()
    .Enrich.FromLogContext()
    .Destructure.ByMaskingProperties("password",
        "authorization",
        "cookie",
        "set-cookie")
    .CreateLogger();
```

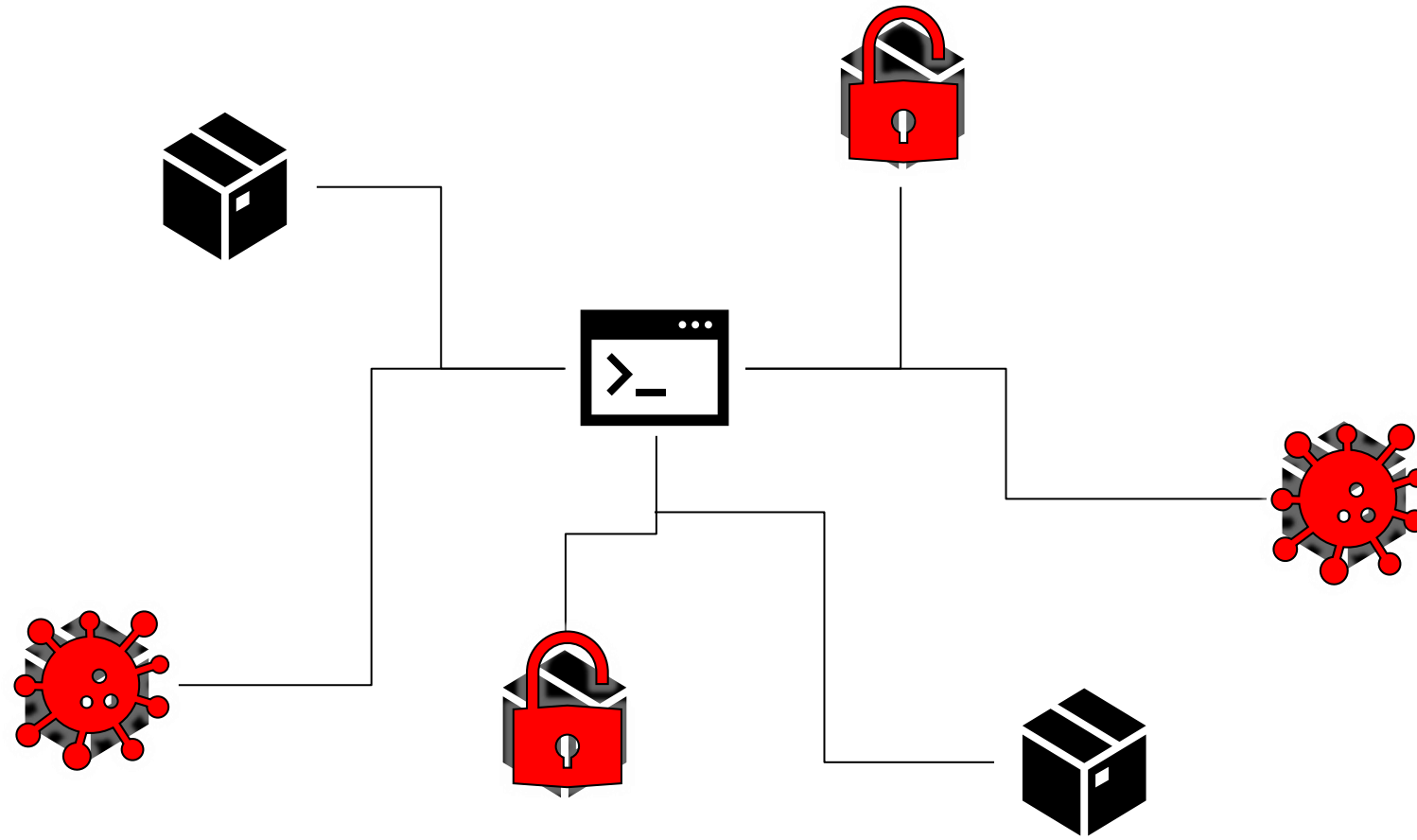
Dependency and Supply-Chain Risks

Common Mistake #7

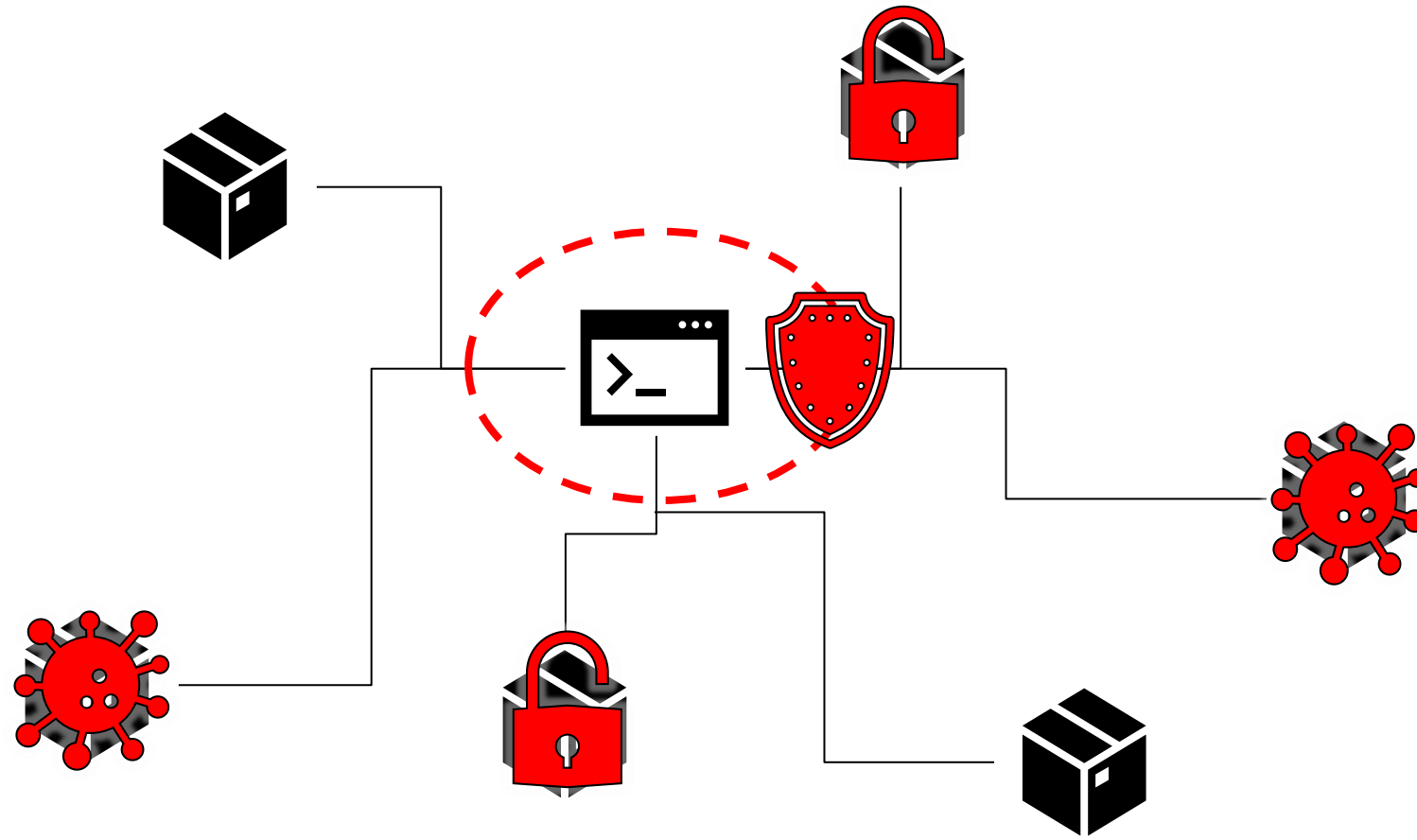
Security



Security



Security



Security

The screenshot shows the NuGet Package Manager interface. On the left, a list of packages is displayed, including Newtonsoft.Json, Newtonsoft.Json.Bson, Newtonsoft.Json.Schema, Microsoft.AspNetCore.Mvc.Newtonsoft.Json, Swashbuckle.AspNetCore.Newtonsoft, GraphQL.Newtonsoft.Json, RestSharp.Newtonsoft.Json, App.Metrics.Formatters.Json, NServiceBus.Newtonsoft.Json, Refit.Newtonsoft.Json, and RestSharp.Serializers.Newtonsoft.Json. On the right, the details for the selected package, Newtonsoft.Json, are shown. The version 12.0.3 is selected. A red circle highlights the 'Vulnerabilities detected (1)' section, which indicates that this package version has at least one vulnerability with high severity. The details pane also shows the package description, version, author, license, date published, project URL, report abuse link, tags, and dependencies.

NuGet Package Manager: Sample

Package source: nuget.org

Newtonsoft.Json

Version: 12.0.3

Vulnerabilities detected (1)

This package version has at least one vulnerability with high severity. It may lead to specific problems in your project. Try updating the package version.

Details

Advisory (high severity): <https://github.com/advisories/GHSA-5crp-9r3c-p9vr>

Options

Description

Json.NET is a popular high-performance JSON framework for .NET

Version: 12.0.3

Author(s): James Newton-King

License: MIT

Date published: Friday, November 8, 2019 (11/8/2019)

Project URL: <https://www.newtonsoft.com/json>

Report Abuse: <https://www.nuget.org/packages/Newtonsoft.Json/12.0.3/ReportAbuse>

Tags: json

Dependencies

- .NETFramework,Version=v2.0
 - No dependencies
- .NETFramework,Version=v3.5
 - No dependencies
- .NETFramework,Version=v4.0
 - No dependencies
- .NETFramework,Version=v4.5
 - No dependencies
- .NETPortable,Version=v0.0,Profile=Profile259
 - No dependencies
- .NETPortable,Version=v0.0,Profile=Profile328
 - No dependencies
- .NETStandard,Version=v1.0
 - Microsoft.CSharp (>= 4.3.0)
 - NETStandard.Library (>= 1.6.1)
 - System.ComponentModel.TypeConverter (>= 4.3.0)

Security

`dotnet list package --vulnerable`

```
jonathantower@MacBookPro eShopCoreMVC % dotnet list package --vulnerable

The following sources were used:
  https://api.nuget.org/v3/index.json

Project `eShopCoreMVC` has the following vulnerable packages
[net6.0]:
Top-level Package      Requested  Resolved  Severity  Advisory URL
> Yarp.ReverseProxy    2.0.0     2.0.0     High      https://github.com/advisories/GHSA-jrjw-qgr2-wfcg

jonathantower@MacBookPro eShopCoreMVC %
```

Security

`dotnet list package --vulnerable`

```
jonathantower@MacBookPro eShopCoreMVC % dotnet list package --vulnerable

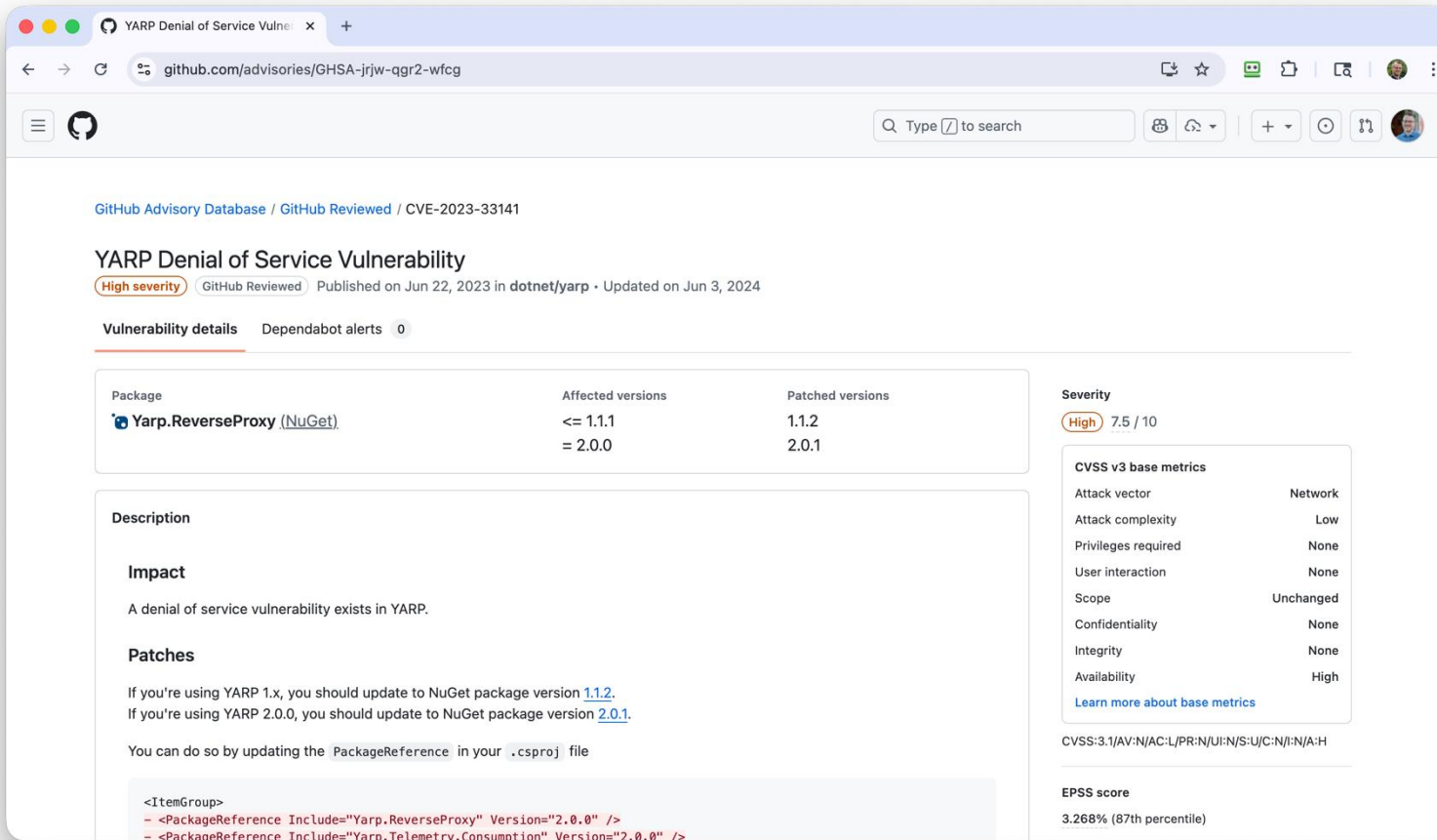
The following sources were used:
https://api.nuget.org/v3/index.json

Project `eShopCoreMVC` has the following vulnerable packages
[net6.0]:
Top-level Package      Requested  Resolved  Severity  Advisory URL
> Yarp.ReverseProxy    2.0.0     2.0.0     High      https://github.com/advisories/GHSA-jrjw-qgr2-wfcg

jonathantower@MacBookPro eShopCoreMVC %
```

Safety & Reliability: Security

dotnet list package --vulnerable



The screenshot shows a web browser displaying a GitHub Advisory Database entry for a Denial of Service (DoS) vulnerability in YARP. The page title is "YARP Denial of Service Vulnerability" with a "High severity" label. It was published on June 22, 2023, in the `dotnet/yarp` repository and updated on June 3, 2024. The vulnerability details table shows that the affected versions are `<= 1.1.1` and `= 2.0.0`, while patched versions are `1.1.2` and `2.0.1`. The severity is rated as "High" with a score of 7.5/10. The CVSS v3 base metrics are listed: Attack vector (Network), Attack complexity (Low), Privileges required (None), User interaction (None), Scope (Unchanged), Confidentiality (None), Integrity (None), and Availability (High). The EPSS score is 3.268% (87th percentile). The description states that a denial of service vulnerability exists in YARP. The patches section provides instructions for updating YARP 1.x to NuGet package version 1.1.2 and YARP 2.0.0 to NuGet package version 2.0.1. A code snippet shows how to update the `PackageReference` in a `.csproj` file.

GitHub Advisory Database / GitHub Reviewed / CVE-2023-33141

YARP Denial of Service Vulnerability

High severity GitHub Reviewed Published on Jun 22, 2023 in `dotnet/yarp` • Updated on Jun 3, 2024

Vulnerability details Dependabot alerts 0

Package	Affected versions	Patched versions
 <code>Yarp.ReverseProxy</code> (NuGet)	<code><= 1.1.1</code> <code>= 2.0.0</code>	<code>1.1.2</code> <code>2.0.1</code>

Description

Impact

A denial of service vulnerability exists in YARP.

Patches

If you're using YARP 1.x, you should update to NuGet package version [1.1.2](#).
If you're using YARP 2.0.0, you should update to NuGet package version [2.0.1](#).

You can do so by updating the `PackageReference` in your `.csproj` file

```
<ItemGroup>
- <PackageReference Include="Yarp.ReverseProxy" Version="2.0.0" />
- <PackageReference Include="Yarp.Telemetry.Consumption" Version="2.0.0" />
```

Severity

High 7.5 / 10

CVSS v3 base metrics

Metric	Value
Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	None
Scope	Unchanged
Confidentiality	None
Integrity	None
Availability	High

[Learn more about base metrics](#)

CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

EPSS score

3.268% (87th percentile)

Safety & Reliability: Security

dotnet list package --vulnerable

The screenshot shows a web browser displaying a GitHub Advisory Database entry for CVE-2023-33141, titled "YARP Denial of Service Vulnerability". The page indicates a "High severity" issue, published on June 22, 2023, in the `dotnet/yarp` repository, and updated on June 3, 2024. The vulnerability details section shows the affected package is `Yarp.ReverseProxy` (NuGet), with affected versions being `<= 1.1.1` and `= 2.0.0`. The patched versions are `1.1.2` and `2.0.1`, which are highlighted with a red circle. The description states that a denial of service vulnerability exists in YARP. The patches section advises updating to NuGet package version 1.1.2 for YARP 1.x and version 2.0.1 for YARP 2.0.0. A code snippet shows how to update the `PackageReference` in a `.csproj` file. On the right, the severity is noted as "High" with a score of 7.5/10, and a table of CVSS v3 base metrics is provided, including attack vector, complexity, privileges required, user interaction, scope, confidentiality, integrity, availability, and network. The EPSS score is 3.268% (87th percentile).

GitHub Advisory Database / GitHub Reviewed / CVE-2023-33141

YARP Denial of Service Vulnerability

High severity GitHub Reviewed Published on Jun 22, 2023 in `dotnet/yarp` • Updated on Jun 3, 2024

Vulnerability details Dependabot alerts 0

Package	Affected versions	Patched versions
<code>Yarp.ReverseProxy</code> (NuGet)	<code><= 1.1.1</code> <code>= 2.0.0</code>	<code>1.1.2</code> <code>2.0.1</code>

Description

Impact

A denial of service vulnerability exists in YARP.

Patches

If you're using YARP 1.x, you should update to NuGet package version [1.1.2](#).
If you're using YARP 2.0.0, you should update to NuGet package version [2.0.1](#).

You can do so by updating the `PackageReference` in your `.csproj` file

```
<ItemGroup>
  - <PackageReference Include="Yarp.ReverseProxy" Version="2.0.0" />
  - <PackageReference Include="Yarp.Telemetry.Consumption" Version="2.0.0" />
```

Severity

High 7.5 / 10

CVSS v3 base metrics

Metric	Value
Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	None
Scope	Unchanged
Confidentiality	None
Integrity	None
Availability	High

[Learn more about base metrics](#)

CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

EPSS score

3.268% (87th percentile)

Security

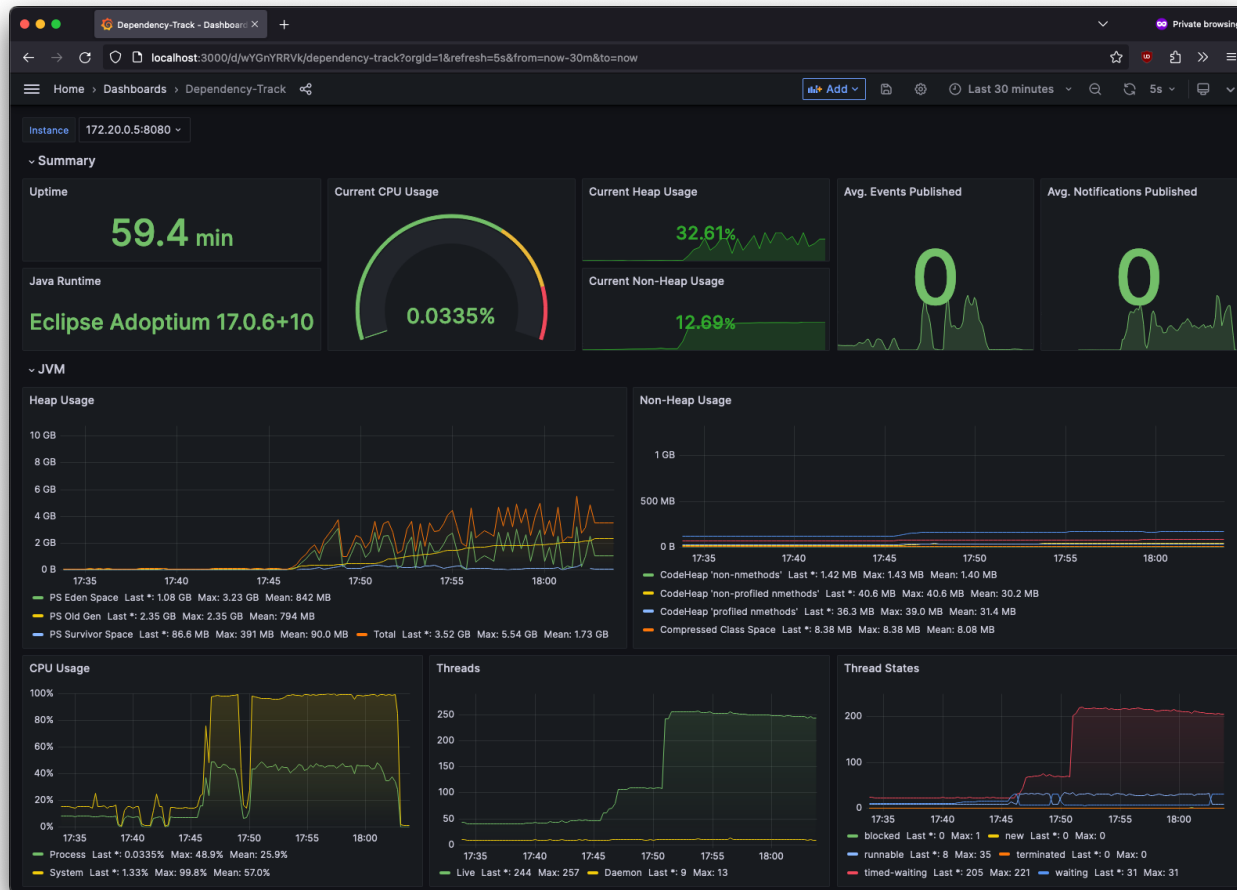
```
# npm audit report

ansi-html *
Severity: high
Uncontrolled Resource Consumption in ansi-html - https://github.com/advisories/GHSA-whgm-jr23-g3j9
fix available via `npm audit fix --force`
Will install @vue/cli-plugin-babel@3.12.1, which is a breaking change
node_modules/ansi-html
  webpack-dev-server 2.0.0-beta - 4.1.0
    Depends on vulnerable versions of ansi-html
    Depends on vulnerable versions of chokidar
    Depends on vulnerable versions of http-proxy-middleware
    Depends on vulnerable versions of yargs
  node_modules/webpack-dev-server
    @vue/cli-service <=5.0.0-beta.3
      Depends on vulnerable versions of @vue/cli-plugin-router
      Depends on vulnerable versions of @vue/cli-shared-utils
      Depends on vulnerable versions of copy-webpack-plugin
      Depends on vulnerable versions of cssnano
      Depends on vulnerable versions of globby
      Depends on vulnerable versions of webpack-dev-server
    node_modules/@vue/cli-service
      @vue/cli-plugin-babel 3.4.0 - 4.5.13
        Depends on vulnerable versions of @vue/cli-service
        Depends on vulnerable versions of @vue/cli-shared-utils
      node_modules/@vue/cli-plugin-babel
        @vue/cli-plugin-router <=4.5.13
          Depends on vulnerable versions of @vue/cli-service
        node_modules/@vue/cli-plugin-router
          @vue/cli-plugin-typescript <=5.0.0-alpha.1
            Depends on vulnerable versions of @vue/cli-service
            Depends on vulnerable versions of @vue/cli-shared-utils
            Depends on vulnerable versions of fork-ts-checker-webpack-plugin
            Depends on vulnerable versions of globby
          node_modules/@vue/cli-plugin-typescript
            @vue/cli-plugin-vuex <=4.5.13
              Depends on vulnerable versions of @vue/cli-service
            node_modules/@vue/cli-plugin-vuex
```

npm audit

Security

OWASP Dependency-Track



Codebase



SBOM Tool



SBOMs
(CycloneDX format)



OWASP Dependency-Track

Transport Layer and Secrets Management

Common Mistake #8

HTTPS and TLS posture



```
builder.WebHost.ConfigureKestrel(o => o.ListenAnyIP(80)); // HTTP only  
app.UseHttpsRedirection();
```

HTTPS and TLS posture



```
builder.WebHost.ConfigureKestrel(o => o.ListenAnyIP(443, l =>
l.UseHttps())));
app.UseHsts();
```

Secrets In Config



```
// appsettings.json
{
  "Jwt": {
    "SigningKey": "secret123" // X
  },
  "Db": {
    "Password": "P@ss!" // X
  }
}
```

Secrets In Config



// Dev: User Secrets

```
dotnet user-secrets set "Db:Password" "... " // not committed to repo
```

// Cloud: Azure Key Vault + Managed Identity

```
using Azure.Identity;
```

```
builder.Configuration
```

```
    .AddAzureKeyVault(new Uri("https://<vault>.vault.azure.net/"),
```

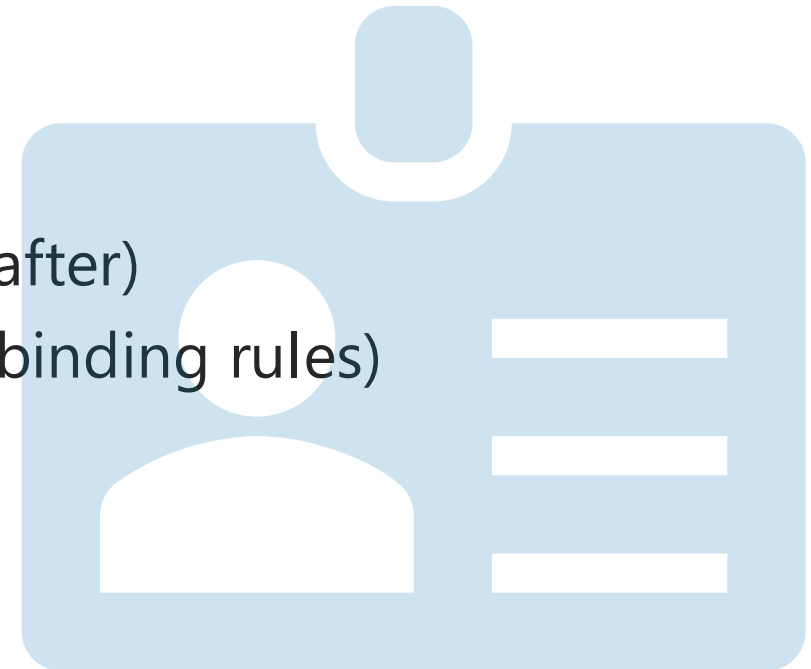
```
        new DefaultAzureCredential()); //  No static
```

key

Best Practices Checklist

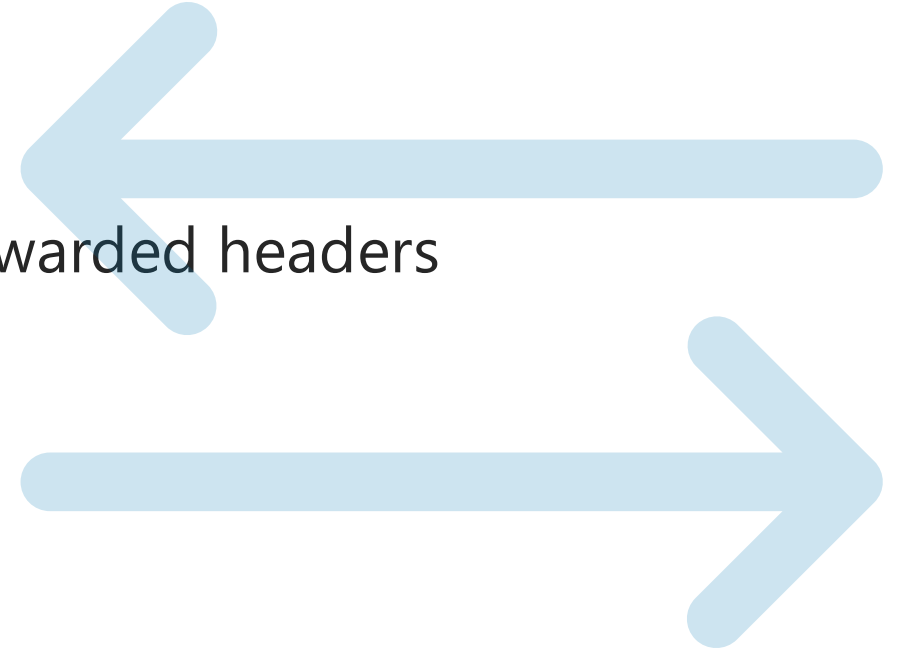
Authentication & Authorization

- ✓ Enforce authorization **centrally**
- ✓ Don't trust **client-supplied identifiers**
- ✓ Validate **object ownership** in queries (not after)
- ✓ Avoid **mass assignment** by default (DTOs, binding rules)



Transport & Infrastructure Defaults

- ✓ Terminate **TLS properly**
- ✓ Use **HSTS** in production
- ✓ Be explicit about **trusted proxies** and forwarded headers



Input & Output Shape

- ✓ Do not **bind entities directly** from requests
- ✓ Reject **unexpected JSON** fields
- ✓ Return **only fields you actually need**
- ✓ Avoid "**general purpose**" APIs



Resource & Abuse Controls

- ✓ Enforce server-side **paging limits**
- ✓ **Rate-limit** sensitive endpoints
- ✓ Cap request **body sizes**



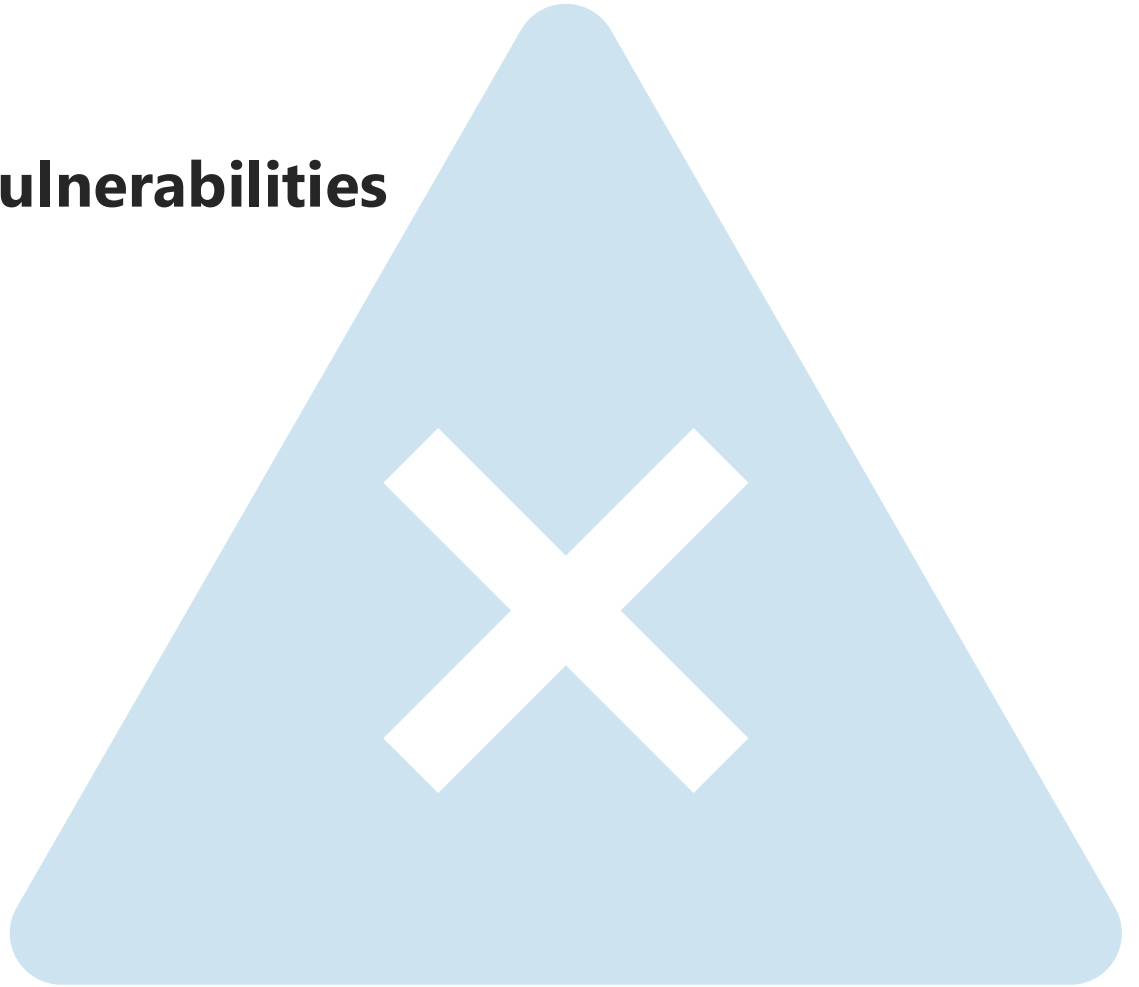
Errors & Logging

- ✓ Never expose **stack traces** in production
- ✓ Use **ProblemDetails** with correlation IDs
- ✓ Redact secrets from logs by default



Dependencies

- ☑ Monitor dependencies for **known vulnerabilities**



Summing Up

- Most API breaches come from **insecure defaults**
- Trust **boundaries** matter more than frameworks
- Authorization bugs are usually **data-shape** bugs
- **General-purpose APIs** are inefficient and leak
- **Unconstrained endpoints** lead to denial-of-service attacks
- Production safety requires **turning things off**
- Security improves when the **secure thing is the easiest thing**



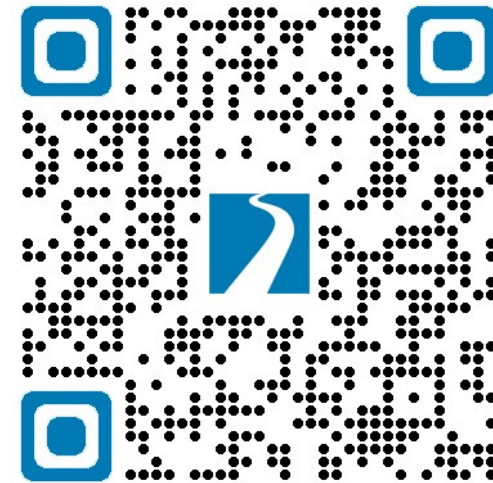
Thanks! Questions?

Jonathan "J." Tower

- 12x Microsoft MVP in .NET
- .NET Foundation Board of Directors
- jtower@trailheadtechnology.com
- trailheadtechnology.com/blog
- [jtowermi](https://twitter.com/jtowermi)
- Jonathan "J." Tower

github.com/trailheadtechnology/api-security

LET'S TALK



bit.ly/th-offer